UNIVERSIDADE PAULISTA

CIÊNCIA DA COMPUTAÇÃO

VIAAGORA: DOCUMENTAÇÃO E CÓDIGO-FONTE
APLICAÇÃO DE SISTEMA DISTRIBUÍDO PARA DISPOSITIVO MÓVEL

Bruno Nascimento de P. Silva – RA G793030

SANTOS-SP

2026

ÍNDICE

1 ORGANIZAÇÃO DAS PASTAS 3
2 COMO EXECUTAR 4
2.1 Requisitos 4
2.2 Subir o servidor 4
2.3 Abrir o site 5
2.4 Ligar o aplicativo 5
2.5 Parar, reiniciar e testar 6
3 EXPERIMENTOS DE FALHA 7
4 REFERÊNCIA DA API 8
5 CÓDIGO-FONTE 9
5.1 Servidor 9
5.2 Banco de dados 18
5.3 Infraestrutura 19
5.4 Aplicativo Android 20

1 ORGANIZAÇÃO DAS PASTAS

O repositório está no GitHub, em github.com/Bruno70i/APS---8-SEMESTRE. Ele reúne o aplicativo, o servidor, o site e a infraestrutura num só lugar, para que qualquer pessoa consiga baixar e executar o projeto inteiro.

```
APS---8-SEMESTRE/
├── android/                  aplicativo Android (Kotlin)
│   └── app/src/main/java/br/com/viaagora/
│       ├── MainActivity.kt          telas, relatos e ajustes
│       ├── RepositorioRemoto.kt     HTTP, cache e WebSocket
│       ├── PainelMapa.kt            mapa (MapLibre + OpenStreetMap)
│       ├── PaginaBastidores.kt      aba Bastidores
│       ├── DiagramaRede.kt          diagrama ao vivo dos processos
│       └── Modelos.kt, EstadoSistema.kt, ComponentesUi.kt, ...
├── server/                   servidor (Node.js)
│   ├── src/                       API, worker e coletor
│   ├── migrations/001.sql         tabelas do PostgreSQL
│   ├── test/                      testes automatizados
│   └── Dockerfile
├── site/                     site do projeto (HTML, CSS e JS)
├── docs/                     Trabalho escrito
├── scripts/build-apk.ps1     compila o APK com os testes
├── compose.yaml              os 7 serviços do ambiente local
├── compose.test.yaml         ambiente isolado dos testes
├── nginx.conf                balanceamento entre as duas APIs
└── README.md
```

Tabela 1 – Os sete serviços do ambiente

Serviço	Imagem	Papel
`nginx`	nginx:1.27-alpine	Porta 8080: site e balanceamento entre as APIs
`api` e `api-2`	código de `server/`	Réplicas da API (HTTP e WebSocket)
`worker`	código de `server/`	Publica os eventos da outbox no RabbitMQ
`collector`	código de `server/`	Lê o painel oficial a cada minuto
`database`	postgres:16-alpine	Banco de dados
`broker`	rabbitmq:4-management-alpine	Mensageria (AMQP)

Fonte: elaborado pelo autor (2026).

2 COMO EXECUTAR

2.1 Requisitos

- **Git**, para baixar o código.
- **Docker Desktop** (ou Docker com Compose), que roda o servidor inteiro. Nada mais precisa ser instalado: Node.js, PostgreSQL, RabbitMQ e Nginx vêm dentro dos contêineres.
- Um **celular Android** (8.0 ou mais novo) ou o emulador do Android Studio, para o aplicativo.

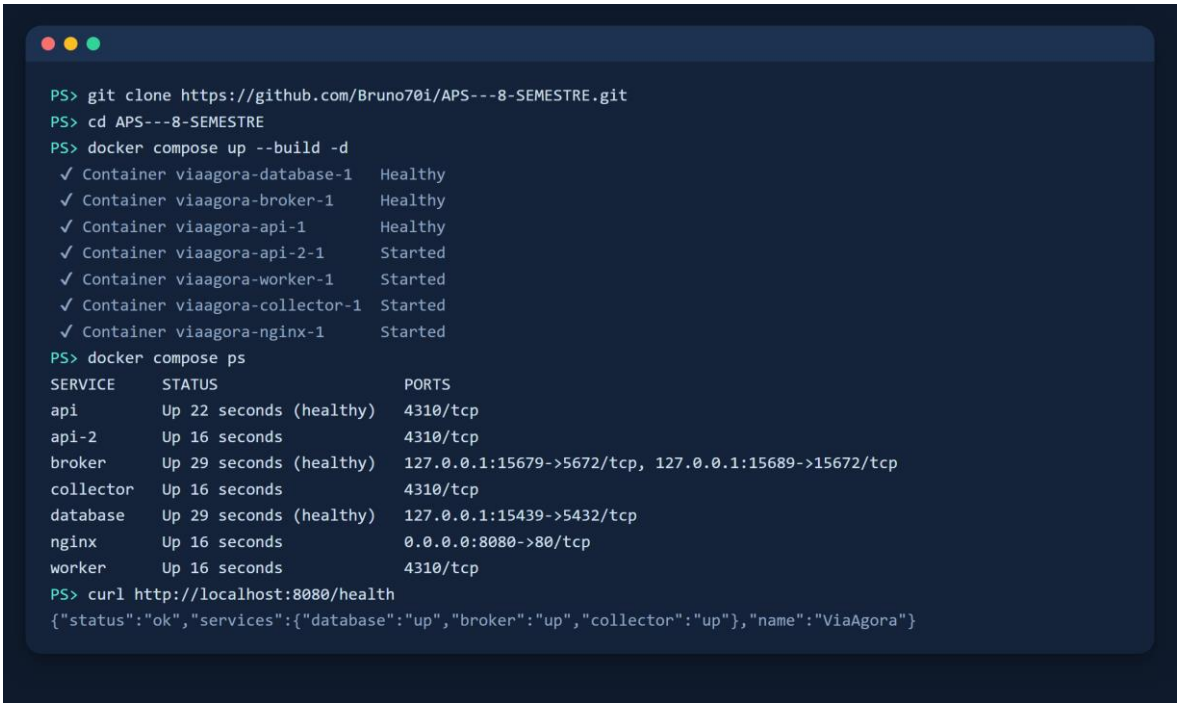
2.2 Subir o servidor

No terminal (PowerShell, no Windows), execute:

```
git clone https://github.com/Bruno70i/APS---8-SEMESTRE.git
cd APS---8-SEMESTRE
docker compose up --build -d
docker compose ps
```

Na primeira vez o Docker baixa as imagens e leva alguns minutos. Quando terminar, os sete serviços aparecem como "Up" (Figura 1). Não é preciso configurar nada: as senhas de desenvolvimento já estão no `compose.yaml`.

Figura 1 – Saída real do terminal ao subir o sistema

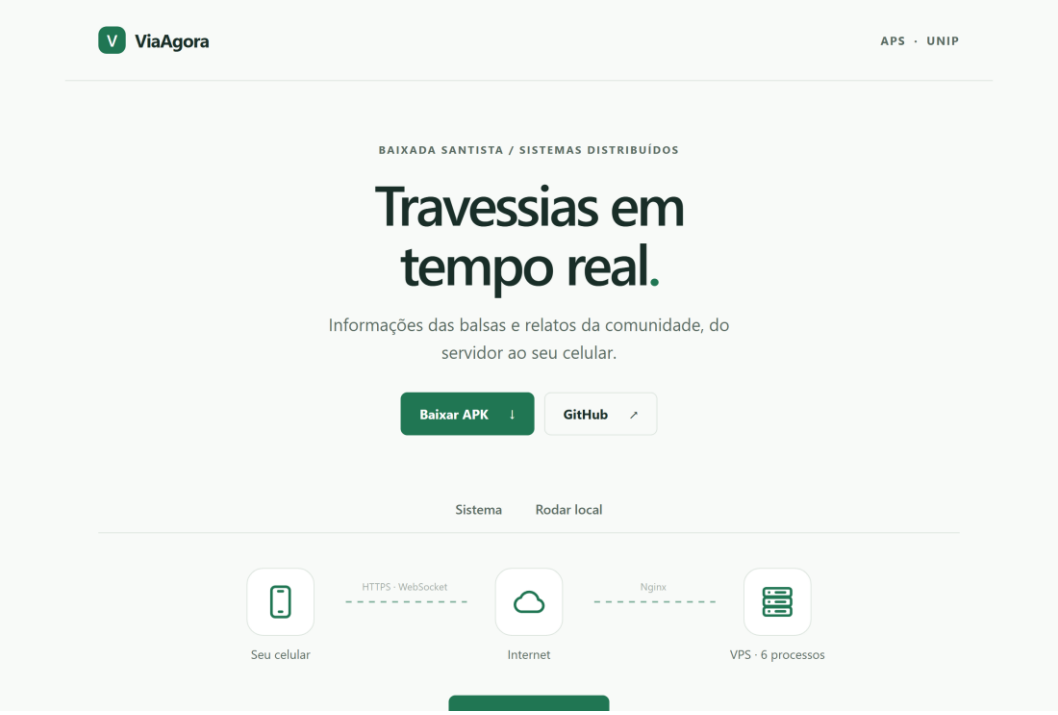


Fonte: elaborado pelo autor (2026).

2.3 Abrir o site

Com o sistema no ar, o endereço `http://localhost:8080` abre o site do projeto, servido pelo Nginx local (Figura 2). Pelo mesmo endereço respondem a API (`/v1/...`) e o estado de cada processo (`/v1/system`).

Figura 2 – Site do projeto rodando no computador (localhost:8080)



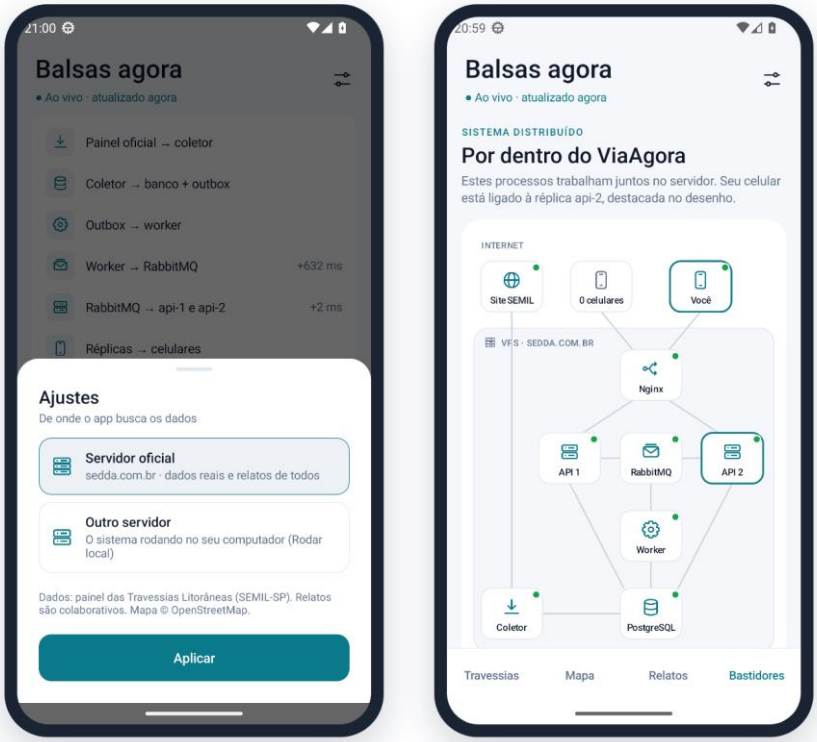
Fonte: elaborado pelo autor (2026).

2.4 Ligar o aplicativo

O APK está em sedda.com.br/downloads/ViaAgora.apk. Ao instalá-lo, o Android pede para permitir a instalação de fontes desconhecidas. O aplicativo abre conectado ao servidor oficial; para usar o sistema do computador, toque no ícone de ajustes e escolha **Outro servidor** (Figura 3):

- no emulador Android: `http://10.0.2.2:8080`;
- no celular: `http://IP_DO_COMPUTADOR:8080`, com o celular na mesma rede Wi-Fi (o IP aparece no comando `ipconfig`). No Windows, permita a porta 8080 na rede privada se o firewall perguntar.

Figura 3 – Ajustes do aplicativo e a aba Bastidores ligada ao servidor



Fonte: elaborado pelo autor (2026).

2.5 Parar, reiniciar e testar

```
 # parar e voltar de onde parou
 docker compose stop
 docker compose start

 # testes do servidor (banco e broker próprios)
 docker compose -f compose.test.yaml up --build --abort-on-container-exit --exit-
code-from tests
 docker compose -f compose.test.yaml down --volumes

 # compilar o aplicativo (JDK 17 e Android SDK 35)
 .\scripts\build-apk.ps1
```

3 EXPERIMENTOS DE FALHA

O ambiente local permite ver o sistema distribuído reagindo a falhas. Os comandos abaixo são executados na pasta do projeto, com o aplicativo aberto na aba Bastidores.

Tabela 2 – Experimentos com o ambiente local

Experimento	Comando	O que observar
Queda de uma réplica	`docker compose stop api`	O Nginx manda tudo para a API 2; o app continua funcionando.
Queda do broker	`docker compose stop broker`	Um relato enviado é salvo e fica "esperando na outbox".
Volta do broker	`docker compose start broker`	O worker publica o evento guardado e os celulares recebem.
Acompanhar os processos	`docker compose logs -f collector worker`	Coletas a cada minuto e eventos publicados.
Estado distribuído	`http://localhost:8080/v1/system`	Sinais de vida, outbox e entregas por réplica.

Fonte: elaborado pelo autor (2026).

Ao terminar, volte o aplicativo para o **Servidor oficial** nos Ajustes. Para apagar todos os dados locais e começar do zero, use `docker compose down --volumes`.

4 REFERÊNCIA DA API

Tabela 3 – Rotas HTTP

Método e rota	Descrição
`POST /v1/participants`	Cria a participação anônima do aparelho. Devolve o token uma única vez; o servidor guarda só o hash SHA-256.
`GET /v1/crossings`	As três travessias com terminais, rota, última leitura oficial, mediana da comunidade por lado e estado da fonte.
`GET /v1/crossings/:id/history?days=7`	Média de espera por hora do dia e melhor horário de cada lado (entre 6h e 22h).
`GET /v1/reports`	Relatos recentes. Filtros: `crossing`, `category`, `status`, `sinceMinutes` e `limit`.
`POST /v1/reports`	Cria um relato. Exige `Authorization: Bearer` e `Idempotency-Key` (UUID).
`PATCH /v1/reports/:id/resolve`	Encerra um relato; só o autor pode.
`GET /v1/system`	Réplica que respondeu, sinais de vida, outbox, contadores e rastro dos eventos.
`GET /health`	Estado do banco, do broker e do coletor.

Fonte: elaborado pelo autor (2026).

Tabela 4 – Mensagens do WebSocket /v1/events

Tipo	Quando chega
`ready`	Ao conectar; informa a réplica e se o broker está conectado.
`resync`	Quando a réplica reconecta ao broker; o app busca tudo de novo.
`broker.status`	Quando o broker cai.
`crossing.updated`	Quando uma leitura oficial muda.
`report.created` / `report.resolved`	Quando um relato é criado ou encerrado.
`source.status`	Quando o painel oficial cai ou volta.

Fonte: elaborado pelo autor (2026).

Exemplo de criação de relato:

```
POST /v1/reports
Authorization: Bearer <token>
Idempotency-Key: 6f1c2a4e-0d7b-4c35-9a8e-2b9d1f7e5c10

{ "crossingId": "santos-guaruja", "side": "A", "category": "fila",
  "waitMinutes": 40, "description": "A fila chega até o canal 6",
  "latitude": -23.98692, "longitude": -46.29379 }
```

5 CÓDIGO-FONTE

Esta seção traz o código-fonte completo do servidor, da infraestrutura e do aplicativo, com nomes e comentários em português e a numeração de linhas dos arquivos do repositório. Os testes automatizados, o site e os arquivos de configuração estão no repositório.

5.1 Servidor

server/src/index.js

```
 1  // Processo da API: cada réplica (api-1 e api-2) roda este arquivo com um NODE_NAME diferente.
 2  import {montarApp} from './app.js';
 3  import {criarPool,migrar} from './db.js';
 4  import {urlRabbit} from './broker.js';
 5  import {nomeDoProcesso,iniciarSinalDeVida} from './heartbeat.js';
 6  const pool=criarPool();
 7  await migrar(pool);
 8  const replica=nomeDoProcesso('api');
 9  const proxiesConfiaveis=process.env.TRUST_PROXY;
10  const app=await
montarApp({pool,urlBroker:urlRabbit(),replica,confiarProxy:proxiesConfiaveis?proxiesConfiaveis.split(',').map(v=>v.tr
im()):false});
11  await app.listen({host:'0.0.0.0',port:Number(process.env.PORT||4310)});
12  // Os nomes clients e broker são lidos pela aba Bastidores do aplicativo.
13  const
sinal=iniciarSinalDeVida(pool,{processo:replica,papel:'api',info:()=>({clients:app.tempoReal.celulares,broker:app.tem
poReal.conectado})});
14  console.log(`ViaAgora: API ${replica} iniciada na porta`,app.server.address().port);
15  let encerrando=false;
16  async function encerrar(){if(encerrando)return;encerrando=true;sinal.parar();await app.close();await pool.end();}
17  process.on('SIGINT',encerrar);process.on('SIGTERM',encerrar);
```

server/src/app.js

```
 1  // API HTTP do ViaAgora. As duas réplicas (api-1 e api-2) rodam este mesmo código e usam o mesmo banco;
 2  // o Nginx reparte os celulares entre elas. As rotas e os nomes dos campos são o contrato com o aplicativo.
 3  import Fastify from 'fastify';
 4  import limiteDeRequisicoes from '@fastify/rate-limit';
 5  import { randomBytes,randomUUID } from 'node:crypto';
 6  import { z } from 'zod';
 7  import {
gerarHash,ErroHttp,esquemaRelato,categorias,salvarRelato,encerrarRelato,listarTravessias,historicoTravessia,estadoFon
te } from './domain.js';
 8  import { idsTravessias } from './crossings.js';
 9  import { paraRelato } from './db.js';
10  import { ligarTempoReal } from './broker.js';
11  import { estadoSistema } from './system.js';
12
13  const uuid=z.string().uuid();
14  // Filtros aceitos em GET /v1/reports e em GET /v1/crossings/:id/history.
15  const
filtrosRelatos=z.object({crossing:z.enum(idsTravessias).optional(),category:z.enum(categorias).optional(),status:z.en
um(['active','resolved']).optional(),
16
sinceMinutes:z.coerce.number().int().min(1).max(10080).default(180),limit:z.coerce.number().int().min(1).max(500).def
ault(200)}).strict();
17  const filtrosHistorico=z.object({days:z.coerce.number().int().min(1).max(30).default(7)}).strict();
18
19  // confiarProxy: atrás do Nginx, o limite de requisições deve contar o IP real de cada usuário, não o do proxy.
20  export async function montarApp({pool,estadoBroker=()=>false,urlBroker=null,replica='api',confiarProxy=false}){
21    const app=Fastify({bodyLimit:8192,logger:false,trustProxy:confiarProxy});
22    // Canal ao vivo (WebSocket) ligado ao RabbitMQ. Nos testes sem broker ele fica desligado.
23    const tempoReal=urlBroker?ligarTempoReal(app,urlBroker,{replica,pool}):null;
24    app.decorate('tempoReal',tempoReal);
25    const brokerConectado=()=>tempoReal?tempoReal.conectado:estadoBroker();
26    await app.register(limiteDeRequisicoes,{max:300,timeWindow:'1 minute'});
27    // Todo erro vira uma mensagem curta em português; falhas internas (ex.: banco fora do ar) respondem 503.
28    app.setErrorHandler((erro,requisicao,resposta)=>{
29      const codigo=erro.statusCode || (erro instanceof z.ZodError?400:503);
30      if(codigo>=500)console.error('Requisição falhou:',erro.message);
31      resposta.code(codigo).send({message:codigo>=500?'Serviço temporariamente indisponível. Tente novamente.':erro
instanceof z.ZodError?'Confira travessia, lado, tipo, espera e coordenadas.':erro.message});
```

```
32    });
33    // Descobre qual aparelho fez o pedido pelo token. O banco guarda só o hash do token.
34    async function autenticar(requisicao){
35      const token=requisicao.headers.authorization?.match(/^Bearer ([A-Za-z0-9_-]{43})$/)?.[1];
36      if(!token)throw new ErroHttp(401,'Identifique este aparelho antes de publicar.');
37      const resultado=await pool.query('SELECT id FROM participants WHERE token_hash=$1',[gerarHash(token)]);
38      if(!resultado.rowCount)throw new ErroHttp(401,'Participação inválida. Reconecte o aplicativo.');
39      return resultado.rows[0].id;
40    }
41    // Participação anônima: o aparelho recebe um token aleatório uma única vez, sem cadastro.
42    app.post('/v1/participants',{config:{rateLimit:{max:15,timeWindow:'1 minute'}}},async(requisicao,resposta)=>{
43      const token=randomBytes(32).toString('base64url'),participantId=randomUUID();
44      await pool.query('INSERT INTO participants(id,token_hash) VALUES($1,$2)',[participantId,gerarHash(token)]);
45      return resposta.code(201).send({participantId,token});
46    });
47    app.get('/v1/crossings',async()=>({crossings:await listarTravessias(pool),source:await
estadoFonte(pool),serverTime:new Date().toISOString()}));
48    app.get('/v1/crossings/:id/history',async requisicao=>{
49      const {days}=filtrosHistorico.parse(requisicao.query);
50      return historicoTravessia(pool,z.string().max(60).parse(requisicao.params.id),days);
51    });
52    app.get('/v1/reports',async requisicao=>{
53      const
filtros=filtrosRelatos.parse(requisicao.query),valores=[filtros.sinceMinutes],condicoes=['created_at>now()-
make_interval(mins=>$1)'];
54      for(const [filtro,coluna] of
[['crossing','crossing_id'],['category','category'],['status','status']])if(filtros[filtro]){valores.push(filtros[fil
tro]);condicoes.push(`${coluna}=$${valores.length}`);}
55      valores.push(filtros.limit);
56      const linhas=await pool.query(`SELECT * FROM reports WHERE ${condicoes.join(' AND ')} ORDER BY created_at
DESC LIMIT $${valores.length}`,valores);
57      return {reports:linhas.rows.map(paraRelato),serverTime:new Date().toISOString()};
58    });
59    // Novo relato: exige o token e uma chave de idempotência (o mesmo envio repetido não duplica o relato).
60    app.post('/v1/reports',async(requisicao,resposta)=>{
61      const autorId=await autenticar(requisicao),entrada=esquemaRelato.parse(requisicao.body);
62      const chave=uuid.safeParse(requisicao.headers['idempotency-key']);
63      if(!chave.success)throw new ErroHttp(400,'Informe uma chave de envio válida.');
64      const resultado=await salvarRelato(pool,autorId,entrada,chave.data,replica);
65      return resposta.code(resultado.reenvio?200:201).send({report:resultado.relato});
66    });
67    app.patch('/v1/reports/:id/resolve',async requisicao=>{
68      const autorId=await autenticar(requisicao),id=uuid.parse(requisicao.params.id);
69      return {report:await encerrarRelato(pool,autorId,id,replica)};
70    });
71    app.get('/v1/system',async()=>estadoSistema(pool,{replica,brokerConectado:brokerConectado()}));
72    // Saúde do serviço: só o banco e o broker derrubam o código HTTP; o coletor é informativo.
73    app.get('/health',async(requisicao,resposta)=>{
74      let banco='up',fonte=null;
75      try{await pool.query('SELECT 1');fonte=await estadoFonte(pool);}catch{banco='down';}
76      const broker=brokerConectado()?'up':'down',ok=banco==='up'&&broker==='up';
77      return
resposta.code(ok?200:503).send({status:ok?'ok':'degraded',services:{database:banco,broker,collector:fonte?.available?
'up':'down'},name:'ViaAgora'});
78    });
79    return app;
80  }
```

server/src/domain.js

```
1  // Regras de negócio do servidor: relatos, leituras do painel oficial e o padrão outbox.
2  // Toda escrita acontece numa transação que grava o dado E o evento juntos: ou salva os dois, ou nenhum.
3  import { createHash, randomUUID } from 'node:crypto';
4  import { z } from 'zod';
5  import { paraRelato,paraLeitura } from './db.js';
6  import { travessias,idsTravessias,travessiaPorId } from './crossings.js';
7
8  export const categorias=['fila','embarque_lento','interrompida','clima','acidente'];
9  export const LEITURA_ATRASADA_MS=10*60_000; // leitura com mais de 10 minutos é mostrada como "dado atrasado"
10
11 // Formato aceito para um novo relato (os nomes dos campos são o contrato com o aplicativo).
12 export const esquemaRelato=z.object({
13   crossingId:z.enum(idsTravessias), side:z.enum(['A','B']), category:z.enum(categorias),
14   waitMinutes:z.number().int().min(0).max(600).nullable().default(null),
15   description:z.string().trim().max(400).default(''),
16   latitude:z.number().min(-90).max(90), longitude:z.number().min(-180).max(180),
17 }).strict();
18 export const gerarHash=valor=>createHash('sha256').update(valor).digest('hex');
19 // Erro com código HTTP (401, 403, 404, 409...) que a API devolve ao aplicativo com a mensagem.
20 export class ErroHttp extends Error
{constructor(statusCode,mensagem){super(mensagem);this.statusCode=statusCode;}}
21
22 // Executa "operacao" dentro de uma transação do PostgreSQL (BEGIN ... COMMIT, ou ROLLBACK se der erro).
23 async function transacao(pool,operacao){
24   const conexao=await pool.connect();
```

```
  try{await conexao.query('BEGIN');const resultado=await operacao(conexao);await conexao.query('COMMIT');return resultado;}
  catch(erro){await conexao.query('ROLLBACK');throw erro;}
  finally{conexao.release();}
 }
 // Grava um evento na outbox, na mesma transação do dado. O worker publica depois no RabbitMQ.
 // "origem" diz quem gerou o evento (api-1, api-2 ou collector), para a aba Bastidores desenhar o caminho.
 async function registrarEvento(conexao,dados,origem){
  const evento={eventId:randomUUID(),...dados,origin:origem};
  await conexao.query('INSERT INTO outbox_events(id,payload) VALUES($1,$2)',[evento.eventId,evento]);
 }

 // Salva um relato e o evento "report.created" juntos. A chave de idempotência (única por participante)
 // faz o reenvio do mesmo relato devolver o relato existente em vez de criar outro.
 export async function salvarRelato(pool,autorId,entrada,chave,origem='api'){
  const hashDoPedido=gerarHash(JSON.stringify(entrada));
  return transacao(pool,async conexao=>{
   const inserido=await conexao.query(`INSERT INTO reports(id,author_id,crossing_id,side,category,wait_minutes,description,latitude,longitude,idempotency_key,request_hash)
    VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11) ON CONFLICT(author_id,idempotency_key) DO NOTHING RETURNING *`,
[randomUUID(),autorId,entrada.crossingId,entrada.side,entrada.category,entrada.waitMinutes,entrada.description,entrada.latitude,entrada.longitude,chave,hashDoPedido]);
   if(inserido.rowCount){
    const relato=paraRelato(inserido.rows[0]);
    await registrarEvento(conexao,{type:'report.created',reportId:relato.id,crossingId:relato.crossingId,version:relato.version},origem);
    return {relato,reenvio:false};
   }
   const existente=(await conexao.query('SELECT * FROM reports WHERE author_id=$1 AND idempotency_key=$2',[autorId,chave])).rows[0];
   if(existente.request_hash!==hashDoPedido)throw new ErroHttp(409,'Esta chave já foi usada para outro relato.');
   // Reenvio do mesmo relato (idempotência): nada é duplicado, só contamos para mostrar na aba Bastidores.
   await conexao.query(`INSERT INTO system_counters(name,value) VALUES('idempotent_replays',1) ON CONFLICT(name) DO UPDATE SET value=system_counters.value+1`);
   return {relato:paraRelato(existente),reenvio:true};
  });
 }
 // Encerra um relato (só quem publicou pode). Encerrar de novo não gera outro evento.
 export async function encerrarRelato(pool,autorId,id,origem='api'){
  return transacao(pool,async conexao=>{
   const resultado=await conexao.query('SELECT * FROM reports WHERE id=$1 FOR UPDATE',[id]);
   if(!resultado.rowCount)throw new ErroHttp(404,'Relato não encontrado.');
   const linha=resultado.rows[0];
   if(linha.author_id!==autorId)throw new ErroHttp(403,'Somente quem publicou pode encerrar este relato.');
   if(linha.status==='resolved')return paraRelato(linha);
   const atualizado=await conexao.query("UPDATE reports SET status='resolved',version=version+1,updated_at=now() WHERE id=$1 RETURNING *",[id]);
   const relato=paraRelato(atualizado.rows[0]);
   await registrarEvento(conexao,{type:'report.resolved',reportId:relato.id,crossingId:relato.crossingId,version:relato.version},origem);
   return relato;
  });
 }

 const mudou=(anterior,nova)=>!anterior||['wait_a','wait_b','boats','operating','weather_a','weather_b','notice'].some(campo=>anterior[campo]!==nova[campo]);
 // Grava uma leitura de cada travessia. O evento só sai quando algo visível mudou, para não acordar os celulares à toa.
 export async function gravarLeituras(pool,leituras,enderecoServico=null){
  return transacao(pool,async conexao=>{
   let mudancas=0;
   for(const l of leituras){
    const nova={wait_a:l.waitA,wait_b:l.waitB,boats:l.boats,operating:l.operating,weather_a:l.weatherA,weather_b:l.weatherB,notice:l.notice};
    const anterior=(await conexao.query('SELECT * FROM crossing_snapshots WHERE crossing_id=$1 ORDER BY collected_at DESC LIMIT 1',[l.crossingId])).rows[0];
    await conexao.query(`INSERT INTO crossing_snapshots(crossing_id,wait_a,wait_b,boats,operating,weather_a,weather_b,period,notice,source,source_time)
VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,'oficial',$10)`,[l.crossingId,l.waitA,l.waitB,l.boats,l.operating,l.weatherA,l.weatherB,l.period,l.notice,l.sourceTime]);
    if(mudou(anterior,nova)){mudancas++;await registrarEvento(conexao,{type:'crossing.updated',crossingId:l.crossingId},'collector');}
   }
   const estado=(await conexao.query('SELECT failures,last_success_at FROM collector_state WHERE id=1 FOR UPDATE')).rows[0];
```

```
      await conexao.query(`INSERT INTO
collector_state(id,mode,feed_url,last_attempt_at,last_success_at,last_error,failures)
VALUES(1,'oficial',$1,now(),now(),NULL,0)
        ON CONFLICT(id) DO UPDATE SET
feed_url=COALESCE($1,collector_state.feed_url),last_attempt_at=now(),last_success_at=now(),last_error=NULL,failures=0
`,[enderecoServico]);
      // A fonte oficial voltou depois de falhar: avisa os celulares.
      if(estado?.failures>0)await registrarEvento(conexao,{type:'source.status',available:true},'collector');
      return {gravadas:leituras.length,mudancas};
    });
  }
  // Registra uma falha da coleta. Na primeira falha seguida, avisa os celulares que a fonte caiu.
  export async function registrarFalha(pool,mensagem){
    return transacao(pool,async conexao=>{
      const estado=(await conexao.query(`INSERT INTO collector_state(id,mode,last_attempt_at,last_error,failures)
VALUES(1,'oficial',now(),$1,1)
        ON CONFLICT(id) DO UPDATE SET last_attempt_at=now(),last_error=$1,failures=collector_state.failures+1
RETURNING failures`,[String(mensagem).slice(0,300)])).rows[0];
      if(estado.failures===1)await registrarEvento(conexao,{type:'source.status',available:false},'collector');
      return estado.failures;
    });
  }

  // Situação da fonte oficial: disponível se a última coleta deu certo há menos de 10 minutos.
  export async function estadoFonte(pool){
    const linha=(await pool.query('SELECT * FROM collector_state WHERE id=1')).rows[0];
    if(!linha)return {mode:null,available:false,lastSuccessAt:null,lastError:'Coletor ainda não executou.'};
    const recente=linha.last_success_at&&Date.now()-linha.last_success_at.getTime()<LEITURA_ATRASADA_MS;
    return
{mode:linha.mode,available:Boolean(recente)&&linha.failures===0,lastSuccessAt:linha.last_success_at?.toISOString()??n
ull,lastError:linha.last_error};
  }
  // As três travessias com a última leitura oficial, a estimativa da comunidade e a contagem de relatos.
  export async function listarTravessias(pool){
    const ultimas=(await pool.query('SELECT DISTINCT ON (crossing_id) * FROM crossing_snapshots ORDER BY
crossing_id,collected_at DESC')).rows;
    // Estimativa da comunidade: mediana das esperas informadas nos últimos 45 minutos, por lado.
    const comunidade=(await pool.query(`SELECT crossing_id,side,percentile_cont(0.5) WITHIN GROUP (ORDER BY
wait_minutes) AS median,count(*)::int AS reports
      FROM reports WHERE status='active' AND wait_minutes IS NOT NULL AND created_at>now()-interval '45 minutes'
GROUP BY crossing_id,side`)).rows;
    // Relatos de "travessia parada" dos últimos 45 minutos sinalizam paralisação antes de o painel oficial mudar.
    const ativos=(await pool.query(`SELECT crossing_id,count(*)::int AS total,
        count(*) FILTER (WHERE category='interrompida' AND created_at>now()-interval '45 minutes')::int AS stops
      FROM reports WHERE status='active' AND created_at>now()-interval '3 hours' GROUP BY crossing_id`)).rows;
    return travessias.map(t=>{
      const leitura=ultimas.find(l=>l.crossing_id===t.id);
      const relatos=ativos.find(r=>r.crossing_id===t.id);
      const lado=letra=>{const m=comunidade.find(r=>r.crossing_id===t.id&&r.side===letra);return
m?{waitMinutes:Math.round(m.median),reports:m.reports}:null;};
      return {...t,status:leitura?paraLeitura(leitura,Date.now()-
leitura.collected_at.getTime()>LEITURA_ATRASADA_MS):null,
        community:{A:lado('A'),B:lado('B')},activeReports:relatos?.total??0,stopReports:relatos?.stops??0};
    });
  }
  // Média de espera por hora do dia nos últimos dias e o melhor horário de cada lado.
  export async function historicoTravessia(pool,id,dias){
    if(!travessiaPorId(id))throw new ErroHttp(404,'Travessia não encontrada.');
    const porHora=(await pool.query(`SELECT extract(hour FROM collected_at AT TIME ZONE 'America/Sao_Paulo')::int
AS hour,
        round(avg(wait_a))::int AS wait_a,round(avg(wait_b))::int AS wait_b,count(*)::int AS samples
      FROM crossing_snapshots WHERE crossing_id=$1 AND operating AND collected_at>now()-make_interval(days=>$2)
GROUP BY 1 ORDER BY 1`,[id,dias])).rows;
    const recentes=(await pool.query(`SELECT to_timestamp(floor(extract(epoch FROM collected_at)/900)*900) AS
bucket,round(avg(wait_a))::int AS wait_a,round(avg(wait_b))::int AS wait_b
      FROM crossing_snapshots WHERE crossing_id=$1 AND operating AND collected_at>now()-interval '6 hours' GROUP
BY 1 ORDER BY 1`,[id])).rows;
    // Leituras com a travessia parada ficam fora das médias: espera zero com balsa parada não é horário bom.
    // Melhor horário só entre 6h e 22h: recomendar a madrugada não ajuda quem precisa atravessar.
    const uteis=porHora.filter(h=>h.hour>=6&&h.hour<=22);
    const melhor=campo=>uteis.length?uteis.reduce((a,b)=>b[campo]<a[campo]?b:a).hour:null;
    return {crossingId:id,days:dias,
      hourly:porHora.map(h=>({hour:h.hour,waitA:h.wait_a,waitB:h.wait_b,samples:h.samples})),
      recent:recentes.map(r=>({at:r.bucket.toISOString(),waitA:r.wait_a,waitB:r.wait_b})),
      bestHour:{A:melhor('wait_a'),B:melhor('wait_b')}};
  }
```

server/src/broker.js

```
// Mensageria do sistema distribuído:
//  - publicarLote (usado pelo worker): lê a outbox e publica os eventos no RabbitMQ, com confirmação;
//  - ligarTempoReal (usado por cada réplica da API): recebe os eventos do RabbitMQ e avisa os celulares pelo
WebSocket.
import amqp from 'amqplib';
import { WebSocketServer,WebSocket } from 'ws';
```

```
 6
 7  const exchange='viaagora.changes'; // exchange do tipo fanout: entrega uma cópia de cada evento a todas as
réplicas
 8  export const urlRabbit=()=>process.env.RABBITMQ_URL || 'amqp://viaagora:viaagora_local_only@127.0.0.1:15679';
 9  const fecharComCalma=async recurso=>{try{await recurso?.close();}catch{}};
10
11  export function ligarTempoReal(app,url,{replica='api',pool=null}={}){
12    const servidorWs=new WebSocketServer({noServer:true,maxPayload:1024});
13    let conexao,canal,temporizador,encerrando=false,conectado=false;
14    // Envia o evento a todos os celulares ligados nesta réplica (quem está com a fila de envio cheia é
desconectado).
15    const avisarCelulares=evento=>{for(const celular of
servidorWs.clients){if(celular.readyState===WebSocket.OPEN){if(celular.bufferedAmount>1024*1024)celular.terminate();e
lse celular.send(JSON.stringify(evento));}}};
16    app.server.on('upgrade',(requisicao,socket,cabeca)=>{
17      if(requisicao.url!=='/v1/events'){socket.destroy();return;}
18      servidorWs.handleUpgrade(requisicao,socket,cabeca,celular=>servidorWs.emit('connection',celular,requisicao));
19    });
20    servidorWs.on('connection',celular=>{
21      celular.alive=true;
22      celular.on('error',()=>{celular.terminate();});
23      celular.on('pong',()=>{celular.alive=true;});
24      // Primeira mensagem: diz ao celular qual réplica o atende e se o broker está conectado.
25      celular.send(JSON.stringify({type:'ready',connected:conectado,replica}));
26    });
27    // A cada 15 s confere se cada celular ainda responde (ping/pong); quem não respondeu é desligado.
28    const verificacao=setInterval(()=>{for(const celular of
servidorWs.clients){if(!celular.alive)celular.terminate();else{celular.alive=false;celular.ping();}}},15000);
29    verificacao.unref();
30    function tentarDeNovo(){
31      if(conectado&&!encerrando)avisarCelulares({type:'broker.status',connected:false});
32      conectado=false;
33      if(!encerrando&&!temporizador){temporizador=setTimeout(()=>{temporizador=null;void
conectar();},1500);temporizador.unref();}
34    }
35    async function conectar(){
36      try{
37        conexao=await amqp.connect(url,{timeout:3000});
38        conexao.on('error',()=>{});conexao.on('close',tentarDeNovo);
39        if(encerrando){await fecharComCalma(conexao);return;}
40        canal=await conexao.createChannel();canal.on('error',()=>{});
41        await canal.assertExchange(exchange,'fanout',{durable:true});
42        // Fila exclusiva desta réplica, ligada à exchange fanout: assim TODAS as réplicas recebem cada evento.
43        const fila=await canal.assertQueue('',{exclusive:true,autoDelete:true});
44        await canal.bindQueue(fila.queue,exchange,'');
45        await canal.consume(fila.queue,mensagem=>{
46          if(!mensagem)return;
47          try{
48            const evento=JSON.parse(mensagem.content.toString());
49            // Cada réplica carimba quem entregou e quando, e registra a entrega para a aba Bastidores.
50            avisarCelulares({...evento,via:replica,deliveredAt:new Date().toISOString()});canal.ack(mensagem);
51            if(pool&&evento.eventId)pool.query('INSERT INTO event_deliveries(event_id,replica,clients)
VALUES($1,$2,$3) ON CONFLICT DO NOTHING',
52              [evento.eventId,replica,servidorWs.clients.size]).catch(()=>{});
53          }catch{canal.nack(mensagem,false,false);}
54        });
55        conectado=true;
56        // Reconectou ao broker: pede aos celulares que busquem tudo de novo (algum evento pode ter passado).
57        avisarCelulares({type:'resync',connected:true});
58      }catch{await fecharComCalma(conexao);tentarDeNovo();}
59    }
60    app.addHook('onReady',async()=>{await conectar();});
61    app.addHook('preClose',async()=>{
62      encerrando=true;conectado=false;clearTimeout(temporizador);clearInterval(verificacao);
63      for(const celular of servidorWs.clients)celular.terminate();servidorWs.close();
64      await fecharComCalma(canal);await fecharComCalma(conexao);
65    });
66    return {get conectado(){return conectado;},get celulares(){return servidorWs.clients.size;}};
67  }
68
69  // Publica até 25 eventos pendentes da outbox. FOR UPDATE SKIP LOCKED garante que dois workers nunca peguem o
mesmo
70  // evento; o evento só é marcado como publicado depois que o RabbitMQ confirma. Se algo falhar, a transação é
71  // desfeita e os eventos continuam pendentes para a próxima tentativa.
72  export async function publicarLote(pool,url=urlRabbit()){
73    const conexaoBanco=await pool.connect();let conexao,canal;
74    try{
75      await conexaoBanco.query('BEGIN');
76      const pendentes=await conexaoBanco.query('SELECT id,payload FROM outbox_events WHERE published_at IS NULL
ORDER BY created_at LIMIT 25 FOR UPDATE SKIP LOCKED');
77      if(!pendentes.rowCount){await conexaoBanco.query('COMMIT');return 0;}
78      conexao=await amqp.connect(url,{timeout:2000});conexao.on('error',()=>{});
79      canal=await conexao.createConfirmChannel();canal.on('error',()=>{});
80      await canal.assertExchange(exchange,'fanout',{durable:true});
81      for(const linha of pendentes.rows){
```

```
      // O consumidor pode receber antes de a confirmação chegar ao worker. Medimos
      // a saída com o relógio do banco, mas só encerramos a pendência após confirmar.
      const publicadoEm=(await conexaoBanco.query('SELECT clock_timestamp() AS sent_at')).rows[0].sent_at;
      await new
Promise((resolver,rejeitar)=>canal.publish(exchange,'',Buffer.from(JSON.stringify(linha.payload)),{persistent:true,me
ssageId:linha.id,contentType:'application/json'},erro=>erro?rejeitar(erro):resolver()));
      await conexaoBanco.query('UPDATE outbox_events SET published_at=$2,attempts=attempts+1 WHERE
id=$1',[linha.id,publicadoEm]);
    }
    await conexaoBanco.query('COMMIT');return pendentes.rowCount;
  }catch(erro){await conexaoBanco.query('ROLLBACK');throw erro;}
  finally{await fecharComCalma(canal);await fecharComCalma(conexao);conexaoBanco.release();}
}
```

server/src/worker.js

```
// Processo worker: a cada 0,75 s publica no RabbitMQ os eventos que estão esperando na outbox.
import {criarPool,migrar} from './db.js';
import {publicarLote} from './broker.js';
import {nomeDoProcesso,iniciarSinalDeVida} from './heartbeat.js';
const pool=criarPool();await migrar(pool);let encerrando=false;
process.on('SIGINT',()=>{encerrando=true;});process.on('SIGTERM',()=>{encerrando=true;});
let publicados=0,ultimaPublicacao=null,voltas=0;
const
sinal=iniciarSinalDeVida(pool,{processo:nomeDoProcesso('worker'),papel:'worker',info:()=>({published:publicados,lastP
ublishAt:ultimaPublicacao})});
console.log('ViaAgora: worker de eventos iniciado.');
while(!encerrando){
  // Se o broker estiver fora do ar, a publicação falha e os eventos continuam guardados na outbox.
  try{const quantidade=await publicarLote(pool);if(quantidade){publicados+=quantidade;ultimaPublicacao=new
Date().toISOString();console.log('Eventos publicados:',quantidade);}}
  catch(erro){console.error('Publicação adiada; dados preservados:',erro.code||erro.name);}
  // Limpeza simples, uma vez por hora: eventos já publicados há mais de 7 dias.
  if(++voltas%4800===0)await pool.query("DELETE FROM outbox_events WHERE published_at<now()-interval '7 days';
DELETE FROM event_deliveries WHERE delivered_at<now()-interval '7 days'").catch(()=>{});
  await new Promise(resolver=>setTimeout(resolver,750));
}
sinal.parar();await pool.end();
```

server/src/collect.js

```
// Uma coleta do painel oficial. Descobre o endereço do serviço varrendo o site (só quando ainda não sabe ou
// quando a leitura falha), lê os dados, normaliza e grava. Falhas ficam registradas e o último dado bom
// continua disponível na API.
import { descobrirEnderecoServico,buscarPainelOficial,ENDERECO_SERVICO_PADRAO } from './crossings.js';
import { gravarLeituras,registrarFalha } from './domain.js';

export function criarColetor({pool,buscarImpl=fetch,enderecoServico=null,log=console}){
  let endereco=enderecoServico;
  async function resolverEndereco(forcar){
    if(endereco&&!forcar)return endereco;
    try{endereco=await descobrirEnderecoServico(buscarImpl);log.info?.('Serviço das travessias encontrado no
site:',endereco);}
    catch(erro){log.warn?.('Varredura do site falhou, usando endereço
conhecido:',erro.message);endereco=endereco||ENDERECO_SERVICO_PADRAO;}
    return endereco;
  }
  return {
    get enderecoServico(){return endereco;},
    async executar(){
      try{
        let leituras;
        // Se a leitura falhar com o endereço guardado, varre o site de novo (o governo pode ter trocado o
endereço).
        try{leituras=await buscarPainelOficial(buscarImpl,await resolverEndereco(false));}
        catch{leituras=await buscarPainelOficial(buscarImpl,await resolverEndereco(true));}
        if(!leituras.length)throw new Error('Nenhuma travessia da Baixada Santista na resposta da fonte.');
        return await gravarLeituras(pool,leituras,endereco);
      }catch(erro){await registrarFalha(pool,erro.message);throw erro;}
    },
  };
}
```

server/src/collector.js

```
// Processo coletor: lê o painel oficial a cada minuto, um único processo para todos os usuários.
import {criarPool,migrar} from './db.js';
import {criarColetor} from './collect.js';
import {nomeDoProcesso,iniciarSinalDeVida} from './heartbeat.js';
const pool=criarPool();await migrar(pool);let encerrando=false;
process.on('SIGINT',()=>{encerrando=true;});process.on('SIGTERM',()=>{encerrando=true;});
const intervalo=Math.max(30_000,Number(process.env.COLLECT_INTERVAL_MS||60_000));
const coletor=criarColetor({pool,enderecoServico:process.env.TRAVESSIAS_FEED_URL||null});
console.log(`ViaAgora: coletor de travessias iniciado (a cada ${intervalo/1000}s).`);
```

```
  let falhas=0,coletas=0,ultimaColeta=null,ultimaDeuCerto=null;
  // Os nomes lastRunAt, lastOk e failures são lidos pela aba Bastidores do aplicativo.
  const
sinal=iniciarSinalDeVida(pool,{processo:nomeDoProcesso('collector'),papel:'collector',info:()=>({lastRunAt:ultimaColeta,lastOk:ultimaDeuCerto,failures:falhas})});
  while(!encerrando){
    try{const {gravadas,mudancas}=await coletor.executar();falhas=0;ultimaDeuCerto=true;console.log(`Coleta: ${gravadas} travessias, ${mudancas} com mudança.`);}
    catch(erro){falhas++;ultimaDeuCerto=false;console.error('Coleta falhou; último dado preservado:',erro.message);}
    ultimaColeta=new Date().toISOString();
    if(++coletas%60===0)await pool.query("DELETE FROM crossing_snapshots WHERE collected_at<now()-interval '30 days'").catch(()=>{});
    // Em falha, espera mais a cada tentativa (até 10 min) para não sobrecarregar a fonte oficial.
    const espera=Math.min(intervalo*2**Math.min(falhas,4),600_000);
    for(let esperado=0;esperado<espera&&!encerrando;esperado+=500)await new Promise(resolver=>setTimeout(resolver,500));
  }
  sinal.parar();await pool.end();
```

server/src/crossings.js

```
  // Travessias acompanhadas e leitura do painel oficial da SEMIL-SP (web scraping).
  import { z } from 'zod';

  // Travessias da Baixada Santista acompanhadas pelo ViaAgora. Terminais e rotas tirados das linhas de balsa do
  // OpenStreetMap (vias 48452393, 496378138 e 466892175); a rota vai do ponto A ao ponto B.
  // Os nomes dos campos (name, pointA, route...) são enviados ao aplicativo e fazem parte da API.
  export const travessias=[
    {id:'santos-guaruja',name:'Santos ⇄ Guarujá',kind:'veiculos',
      pointA:{name:'Santos',address:'Praça Gago Coutinho, 525 - Ponta da Praia',latitude:-23.98692,longitude:-46.29379},
      pointB:{name:'Guarujá',address:'Av. Ademar de Barros, 3300',latitude:-23.98832,longitude:-46.28944},
      route:[[-23.98692,-46.29379],[-23.98832,-46.28944]]},
    {id:'santos-vicente-de-carvalho',name:'Santos ⇄ Vicente de Carvalho',kind:'pedestres',
      pointA:{name:'Santos',address:'Praça da República - Centro',latitude:-23.93171,longitude:-46.32451},
      pointB:{name:'Vicente de Carvalho',address:'Av. Thiago Ferreira',latitude:-23.93964,longitude:-46.30827},
      route:[[-23.93171,-46.32451],[-23.93075,-46.32423],[-23.93068,-46.32406],[-23.93152,-46.31825],[-23.93313,-46.31383],
        [-23.93512,-46.31194],[-23.94001,-46.31023],[-23.93963,-46.30835],[-23.93964,-46.30827]]},
    {id:'bertioga-guaruja',name:'Bertioga ⇄ Guarujá',kind:'veiculos',
      pointA:{name:'Bertioga',address:'Av. Henrique Costabile, 50 - Jardim Veleiros',latitude:-23.85639,longitude:-46.14465},
      pointB:{name:'Guarujá',address:'Estrada Guarujá-Bertioga, km 29',latitude:-23.86028,longitude:-46.14337},
      route:[[-23.85639,-46.14465],[-23.86028,-46.14337]]},
  ];
  export const idsTravessias=travessias.map(t=>t.id);
  export const travessiaPorId=id=>travessias.find(t=>t.id===id);

  export const ENDERECO_SITE='https://semil.sp.gov.br/travessias/';
  // Endereço conhecido do serviço do painel, usado se a varredura do site falhar.
  export const ENDERECO_SERVICO_PADRAO='https://drivetis-travessias.brazilsouth.cloudapp.azure.com:1444/TravessiaService/PT';
  const cabecalhos={'User-Agent':'Mozilla/5.0 (compatible; ViaAgora-APS/1.0; projeto academico)','Accept':'text/html,application/xhtml+xml,application/json','Accept-Language':'pt-BR,pt;q=0.9'};

  // "Santos Guarujá" -> "santos-guaruja": sem acentos, minúsculo e com hífens.
  export const gerarSlug=valor=>valor.normalize('NFD').replace(/[̀-ͯ]/g,'').toLowerCase().trim().replace(/[^a-z0-9]+/g,'-').replace(/^-|-$/g,'');
  // Aviso fixo do 0800 que aparece em todas as travessias; não informa nada sobre a situação atual.
  const avisoGenerico=/^(o\s+)?0800[-\d\s]*.{0,40}24 horas/i;
  const removerHtml=valor=>(valor??'').replace(/<[^>]*>/g,' ').replace(/ /g,' ').replace(/\s+/g,' ').trim();

  // Página da SEMIL -> <script ...travessia-service...> -> variável getInfo (ignora linhas comentadas).
  export function encontrarScriptDoPainel(html,base=ENDERECO_SITE){
    const endereco=html.match(/<script[^>]+src="([^"]*travessia-service[^"]*)"/i)?.[1];
    return endereco?new URL(endereco,base).toString():null;
  }
  export function encontrarEnderecoServico(script){
    for(const linha of script.split(/\r?\n/)){
      if(linha.trim().startsWith('//'))continue;
      const endereco=linha.match(/getInfo\s*=\s*["'](https?:\/\/[^"']+)["']/)?.[1];
      if(endereco)return endereco.endsWith('/')?endereco:endereco+'/';
    }
    return null;
  }

  // Baixa uma página ou um JSON, desistindo depois de 15 segundos.
  async function baixar(buscarImpl,endereco,tipo){
    const resposta=await buscarImpl(endereco,{headers:cabecalhos,signal:AbortSignal.timeout(15000)});
    if(!resposta.ok)throw new Error(`Fonte respondeu ${resposta.status} em ${new URL(endereco).host}`);
    return tipo==='json'?resposta.json():resposta.text();
  }
  // Refaz o caminho do navegador: página -> script do painel -> endereço do serviço de dados.
```

```
export async function descobrirEnderecoServico(buscarImpl=fetch){
  const html=await baixar(buscarImpl,ENDERECO_SITE,'text');
  const script=encontrarScriptDoPainel(html);
  if(!script)throw new Error('Script do painel não encontrado na página.');
  const base=encontrarEnderecoServico(await baixar(buscarImpl,script,'text'));
  if(!base)throw new Error('Endereço do serviço não encontrado no script.');
  return base+'PT';
}

// Formato de cada item do JSON do painel oficial (validado com Zod; itens fora do formato são ignorados).
const itemPainel=z.object({
  TravessiaPontoA:z.string(),TravessiaPontoB:z.string(),
  EsperaPontoA:z.number().int().min(0).max(1440),EsperaPontoB:z.number().int().min(0).max(1440),
  NumEmbarca:z.number().int().min(0).max(50),EmbarcaAtivo:z.boolean(),
  ClimaA:z.string().nullish(),ClimaB:z.string().nullish(),DiaNoite:z.string().nullish(),
  AvisoUsuarioApp:z.string().nullish(),AvisoUsuario:z.string().nullish(),DataConsulta:z.string().nullish(),
}).passthrough();

// "10/28/2026 9:16:41 PM" no horário de Brasília (UTC-3, sem horário de verão) -> data ISO em UTC.
export function lerHorarioFonte(valor){
  const m=valor?.match(/^(\d{1,2})\/(\d{1,2})\/(\d{4}) (\d{1,2}):(\d{2}):(\d{2}) (AM|PM)$/);
  if(!m)return null;
  let hora=Number(m[4])%12;if(m[7]==='PM')hora+=12;
  const data=new Date(Date.UTC(Number(m[3]),Number(m[1])-1,Number(m[2]),hora+3,Number(m[5]),Number(m[6])));
  return Number.isNaN(data.getTime())?null:data.toISOString();
}

const limparAviso=valor=>{const texto=removerHtml(valor).slice(0,400);return
texto&&!avisoGenerico.test(texto)?texto:null;};
// O painel nem sempre zera as embarcações numa paralisação; às vezes só avisa no texto. Restrições parciais
// (ex.: só veículos pesados) e avisos de retomada não contam como travessia parada.
const
palavrasDeParada=/\b(interrompid[ao]s?|paralisad[ao]s?|paralisacao|suspens[ao]s?|interditad[ao]s?|fechad[ao]s?|sem
operacao)\b/;
const retomada=/(retomad|normaliz|restabelecid|liberad|reabert|voltou a operar)/;
const parcial=/(pesad|caminh|carga)/;
// Frases que falam da travessia inteira valem mesmo se o aviso também citar veículos pesados.
const paradaTotal=/(travessia (esta |foi )?(interrompid|paralisad|suspens|fechad)|operacao (esta |foi
)?(suspens|interrompid|paralisad)|sem operacao|paralisacao (total|geral))/;
export function ehAvisoDeParada(texto){
  const simples=(texto??'').normalize('NFD').replace(/[̀-ͯ]/g,'').toLowerCase();
  if(retomada.test(simples))return false;
  return paradaTotal.test(simples)||(palavrasDeParada.test(simples)&&!parcial.test(simples));
}
// JSON do painel -> leituras das três travessias da Baixada Santista (as demais travessias do estado são
ignoradas).
export function lerPainel(dados){
  if(!Array.isArray(dados))throw new Error(typeof dados==='string'?`Fonte recusou a consulta: ${dados}`:'Formato
inesperado da fonte.');
  const leituras=[];
  for(const bruto of dados){
    const item=itemPainel.safeParse(bruto);
    if(!item.success)continue;
    const d=item.data,id=gerarSlug(`${d.TravessiaPontoA} ${d.TravessiaPontoB}`);
    if(!travessiaPorId(id))continue;
    const aviso=limparAviso(d.AvisoUsuarioApp||d.AvisoUsuario);

leituras.push({crossingId:id,waitA:d.EsperaPontoA,waitB:d.EsperaPontoB,boats:d.NumEmbarca,operating:d.EmbarcaAtivo&&d
.NumEmbarca>0&&!ehAvisoDeParada(aviso),

weatherA:removerHtml(d.ClimaA)||null,weatherB:removerHtml(d.ClimaB)||null,period:removerHtml(d.DiaNoite)||null,
      notice:aviso,sourceTime:lerHorarioFonte(d.DataConsulta)});
  }
  return leituras;
}

export async function buscarPainelOficial(buscarImpl=fetch,enderecoServico=ENDERECO_SERVICO_PADRAO){
  return lerPainel(await baixar(buscarImpl,enderecoServico,'json'));
}
```

server/src/db.js

```
// Banco de dados: conexão com o PostgreSQL, criação das tabelas e conversão das linhas para o formato da API.
import pg from 'pg';
import { readFile } from 'node:fs/promises';

// Endereço do banco. No Docker vem da variável DATABASE_URL; o valor padrão serve para rodar fora do Docker.
export const enderecoBanco=()=>process.env.DATABASE_URL ||
'postgresql://viaagora:viaagora_local_only@127.0.0.1:15439/viaagora';

// Conjunto de conexões reaproveitadas pelo processo (no máximo 10 abertas ao mesmo tempo).
export function criarPool(){return new
pg.Pool({connectionString:enderecoBanco(),connectionTimeoutMillis:3000,max:10});}

// Cria as tabelas (migração). A trava impede que duas réplicas que sobem juntas rodem a migração ao mesmo tempo.
```

```
export async function migrar(pool){
  const conexao=await pool.connect();
  try{
    await conexao.query('BEGIN');
    await conexao.query('SELECT pg_advisory_xact_lock(43104310)');
    await conexao.query(await readFile(new URL('../migrations/001.sql',import.meta.url),'utf8'));
    await conexao.query('COMMIT');
  }catch(erro){await conexao.query('ROLLBACK');throw erro;}finally{conexao.release();}
}

// Linha da tabela "reports" -> relato no formato JSON da API.
// Os nomes dos campos (authorId, crossingId...) são o contrato com o aplicativo e não podem mudar.
export function paraRelato(linha){return
{id:linha.id,authorId:linha.author_id,crossingId:linha.crossing_id,side:linha.side,category:linha.category,

waitMinutes:linha.wait_minutes,description:linha.description,latitude:linha.latitude,longitude:linha.longitude,status
:linha.status,version:linha.version,
  createdAt:linha.created_at.toISOString(),updatedAt:linha.updated_at.toISOString()};}

// Linha da tabela "crossing_snapshots" (leitura do painel oficial) -> situação da travessia na API.
export function paraLeitura(linha,atrasada=false){return
{waitA:linha.wait_a,waitB:linha.wait_b,boats:linha.boats,operating:linha.operating,
  weatherA:linha.weather_a,weatherB:linha.weather_b,period:linha.period,notice:linha.notice,source:linha.source,

sourceTime:linha.source_time?.toISOString()??null,collectedAt:linha.collected_at.toISOString(),stale:atrasada};}
```

server/src/migrate.js

```
// Comando avulso para criar ou atualizar as tabelas do banco: node src/migrate.js
import {criarPool,migrar} from './db.js';
const pool=criarPool();
try{await migrar(pool);console.log('ViaAgora: banco atualizado.');}finally{await pool.end();}
```

server/src/heartbeat.js

```
// Sinal de vida (heartbeat) dos processos, mostrado na aba Bastidores do aplicativo.
import os from 'node:os';

// Nome do processo (ex.: api-1, api-2, worker, collector), definido no Docker Compose.
export const nomeDoProcesso=papel=>process.env.NODE_NAME||`${papel}-${os.hostname().slice(0,6)}`;

// A cada 5 s o processo grava "estou vivo" no banco. Quem parar de gravar aparece como fora do ar.
// "info" devolve dados extras do processo (por exemplo, quantos celulares estão ligados na réplica).
export function iniciarSinalDeVida(pool,{processo,papel,info=()=>({}),intervaloMs=5000}){
  let primeiro=true;
  const pulsar=async()=>{
    try{
      await pool.query(`INSERT INTO node_heartbeats(node,role,info,started_at,last_seen)
VALUES($1,$2,$3,now(),now())
        ON CONFLICT(node) DO UPDATE SET
role=$2,info=$3,last_seen=now()${primeiro?',started_at=now()':''}`,[processo,papel,info()]);
      primeiro=false;
    }catch{}
  };
  void pulsar();
  const relogio=setInterval(pulsar,intervaloMs);relogio.unref();
  return {pulsar,parar:()=>clearInterval(relogio)};
}
```

server/src/system.js

```
// Estado do sistema distribuído (rota GET /v1/system), usado pela aba Bastidores:
// quais processos estão vivos, quantos eventos esperam na outbox e o caminho dos últimos eventos.
export const VIVO_ATE_MS=15_000; // sem sinal de vida há mais de 15 s, o processo aparece como fora do ar

export async function estadoSistema(pool,{replica,brokerConectado}){
  const processos=(await pool.query(`SELECT node,role,info,started_at,last_seen,(extract(epoch FROM now()-
last_seen)*1000)::int AS age_ms
    FROM node_heartbeats WHERE last_seen>now()-interval '30 minutes' ORDER BY role,node`)).rows;
  const outbox=(await pool.query(`SELECT count(*) FILTER (WHERE published_at IS NULL)::int AS pending,
    count(*) FILTER (WHERE created_at>now()-interval '1 hour')::int AS last_hour FROM outbox_events`)).rows[0];
  const contadores=Object.fromEntries((await pool.query('SELECT name,value::int AS value FROM
system_counters')).rows.map(linha=>[linha.name,linha.value]));
  const coletas=(await pool.query(`SELECT count(*) FILTER (WHERE collected_at>now()-interval '24 hours')::int AS
last24h,count(*)::int AS total FROM crossing_snapshots`)).rows[0];
  // Últimos 8 eventos da outbox com as entregas de cada réplica (quando e para quantos celulares).
  const eventos=(await pool.query(`SELECT o.id,o.payload->>'type' AS type,o.payload->>'origin' AS
origin,o.payload->>'crossingId' AS crossing_id,o.created_at,o.published_at,
      COALESCE(json_agg(json_build_object('replica',d.replica,'clients',d.clients,'deliveredAt',d.delivered_at)
ORDER BY d.replica) FILTER (WHERE d.replica IS NOT NULL),'[]') AS deliveries
    FROM (SELECT * FROM outbox_events ORDER BY created_at DESC LIMIT 8) o LEFT JOIN event_deliveries d ON
d.event_id=o.id
    GROUP BY o.id,o.payload,o.created_at,o.published_at ORDER BY o.created_at DESC`)).rows;
  const apisVivas=processos.filter(p=>p.role==='api'&&p.age_ms<VIVO_ATE_MS);
```

```
    // Os nomes dos campos abaixo são o contrato com o aplicativo.
    return {
      servedBy:replica,serverTime:new Date().toISOString(),aliveAfterMs:VIVO_ATE_MS,

nodes:processos.map(p=>({node:p.node,role:p.role,alive:p.age_ms<VIVO_ATE_MS,ageMs:p.age_ms,startedAt:p.started_at.toI
SOString(),info:p.info})),
      services:{database:'up',broker:brokerConectado||apisVivas.some(p=>p.info?.broker)?'up':'down'},
      outbox:{pending:outbox.pending,lastHour:outbox.last_hour},
      counters:{idempotentReplays:contadores.idempotent_replays??0},
      collections:coletas,

events:eventos.map(e=>({eventId:e.id,type:e.type,origin:e.origin,crossingId:e.crossing_id,createdAt:e.created_at.toIS
OString(),
        publishedAt:e.published_at?.toISOString()??null,
        deliveries:e.deliveries.map(d=>({replica:d.replica,clients:d.clients,deliveredAt:new
Date(d.deliveredAt).toISOString()}))})),
    };
  }
```

5.2 Banco de dados

server/migrations/001.sql

```
CREATE TABLE IF NOT EXISTS participants (
  id UUID PRIMARY KEY,
  token_hash TEXT NOT NULL UNIQUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE IF NOT EXISTS crossing_snapshots (
  id BIGSERIAL PRIMARY KEY,
  crossing_id TEXT NOT NULL,
  wait_a INTEGER NOT NULL CHECK (wait_a BETWEEN 0 AND 1440),
  wait_b INTEGER NOT NULL CHECK (wait_b BETWEEN 0 AND 1440),
  boats INTEGER NOT NULL CHECK (boats BETWEEN 0 AND 50),
  operating BOOLEAN NOT NULL,
  weather_a TEXT,
  weather_b TEXT,
  period TEXT,
  notice TEXT,
  source TEXT NOT NULL CHECK (source IN ('oficial','simulado')),
  source_time TIMESTAMPTZ,
  collected_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS crossing_snapshots_latest ON crossing_snapshots(crossing_id,collected_at DESC);
CREATE TABLE IF NOT EXISTS collector_state (
  id SMALLINT PRIMARY KEY CHECK (id = 1),
  mode TEXT NOT NULL,
  feed_url TEXT,
  last_attempt_at TIMESTAMPTZ,
  last_success_at TIMESTAMPTZ,
  last_error TEXT,
  failures INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE IF NOT EXISTS reports (
  id UUID PRIMARY KEY,
  author_id UUID NOT NULL REFERENCES participants(id),
  crossing_id TEXT NOT NULL CHECK (crossing_id IN ('santos-guaruja','santos-vicente-de-carvalho','bertioga-
guaruja')),
  side CHAR(1) NOT NULL CHECK (side IN ('A','B')),
  category TEXT NOT NULL CHECK (category IN ('fila','embarque_lento','interrompida','clima','acidente')),
  wait_minutes INTEGER CHECK (wait_minutes BETWEEN 0 AND 600),
  description TEXT NOT NULL DEFAULT '' CHECK (length(description) <= 400),
  latitude DOUBLE PRECISION NOT NULL CHECK (latitude BETWEEN -90 AND 90),
  longitude DOUBLE PRECISION NOT NULL CHECK (longitude BETWEEN -180 AND 180),
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active','resolved')),
  version INTEGER NOT NULL DEFAULT 1,
  idempotency_key UUID NOT NULL,
  request_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(author_id,idempotency_key)
);
CREATE INDEX IF NOT EXISTS reports_recent ON reports(crossing_id,created_at DESC);
CREATE TABLE IF NOT EXISTS outbox_events (
  id UUID PRIMARY KEY,
  payload JSONB NOT NULL,
  attempts INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at TIMESTAMPTZ
);
CREATE INDEX IF NOT EXISTS outbox_events_pending ON outbox_events(created_at) WHERE published_at IS NULL;

-- Bastidores: sinal de vida de cada processo, entrega de cada evento por réplica e contadores didáticos.
```

```
CREATE TABLE IF NOT EXISTS node_heartbeats (
  node TEXT PRIMARY KEY,
  role TEXT NOT NULL CHECK (role IN ('api','worker','collector')),
  info JSONB NOT NULL DEFAULT '{}',
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_seen TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE IF NOT EXISTS event_deliveries (
  event_id UUID NOT NULL,
  replica TEXT NOT NULL,
  clients INTEGER NOT NULL DEFAULT 0,
  delivered_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
  PRIMARY KEY(event_id,replica)
);
CREATE INDEX IF NOT EXISTS event_deliveries_recent ON event_deliveries(delivered_at);
CREATE TABLE IF NOT EXISTS system_counters (
  name TEXT PRIMARY KEY,
  value BIGINT NOT NULL DEFAULT 0
);
```

5.3 Infraestrutura

compose.yaml

```
# Ambiente local: o mesmo sistema da VPS, rodando no seu computador.
# Suba tudo com "docker compose up --build -d" e acesse http://localhost:8080.
name: viaagora
x-backend: &backend
  DATABASE_URL: postgresql://viaagora:${DATABASE_PASSWORD:-viaagora_local_only}@database:5432/viaagora
  RABBITMQ_URL: amqp://viaagora:${BROKER_PASSWORD:-viaagora_local_only}@broker:5672
  PORT: 4310
services:
  database:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: viaagora
      POSTGRES_PASSWORD: ${DATABASE_PASSWORD:-viaagora_local_only}
      POSTGRES_DB: viaagora
    ports: ["127.0.0.1:15439:5432"]
    volumes: ["viaagora_database:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U viaagora -d viaagora"]
      interval: 3s
      timeout: 3s
      retries: 30
    restart: unless-stopped
  broker:
    image: rabbitmq:4-management-alpine
    environment:
      RABBITMQ_DEFAULT_USER: viaagora
      RABBITMQ_DEFAULT_PASS: ${BROKER_PASSWORD:-viaagora_local_only}
    ports: ["127.0.0.1:15679:5672", "127.0.0.1:15689:15672"]
    volumes: ["viaagora_broker:/var/lib/rabbitmq"]
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 5s
      timeout: 5s
      retries: 30
    restart: unless-stopped
  api:
    build: ./server
    environment: {<<: *backend, NODE_NAME: api-1}
    depends_on:
      database: {condition: service_healthy}
      broker: {condition: service_healthy}
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "node", "-e",
"fetch('http://127.0.0.1:4310/health').then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))"]
      interval: 5s
      timeout: 3s
      retries: 30
  api-2:
    build: ./server
    environment: {<<: *backend, NODE_NAME: api-2}
    depends_on:
      api: {condition: service_healthy}
    restart: unless-stopped
  worker:
    build: ./server
    command: ["node", "src/worker.js"]
    environment: {<<: *backend, NODE_NAME: worker}
    depends_on:
```

```
      api: {condition: service_healthy}
    restart: unless-stopped
  collector:
    build: ./server
    command: ["node", "src/collector.js"]
    environment: {<<: *backend, NODE_NAME: collector}
    depends_on:
      api: {condition: service_healthy}
    restart: unless-stopped
  # Porta de entrada única: mostra o site e reparte o app entre as duas APIs.
  nginx:
    image: nginx:1.27-alpine
    ports: ["8080:80"]
    volumes: ["./nginx.conf:/etc/nginx/conf.d/default.conf:ro", "./site:/usr/share/nginx/html:ro"]
    depends_on:
      api: {condition: service_healthy}
      api-2: {condition: service_started}
    restart: unless-stopped
volumes:
  viaagora_database:
  viaagora_broker:
```

nginx.conf

```
# Nginx do ambiente local: a mesma ideia da VPS, sem HTTPS.
# O site sai da pasta site/ e tudo em /v1 vai para as duas réplicas da API, alternando entre elas.
upstream api {
    zone api 64k;
    server api:4310 max_fails=1 fail_timeout=30s;
    server api-2:4310 max_fails=1 fail_timeout=30s;
}
map $http_upgrade $connection_upgrade { default upgrade; '' close; }
server {
    listen 80;
    root /usr/share/nginx/html;
    location /v1/ {
        proxy_pass http://api;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_set_header Host $host;
        # Réplica parada: desiste em 2 s e tenta a outra, em vez de deixar o celular esperando.
        proxy_connect_timeout 2s;
        proxy_read_timeout 75s;
        proxy_next_upstream error timeout http_502 http_503 http_504;
    }
    location = /health { proxy_pass http://api; }
}
```

server/Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY src ./src
COPY migrations ./migrations
USER node
EXPOSE 4310
CMD ["node", "src/index.js"]
```

5.4 Aplicativo Android

android/app/src/main/java/br/com/viaagora/MainActivity.kt

```
package br.com.viaagora

import android.Manifest
import android.content.pm.PackageManager
import android.graphics.Color
import android.location.*
import android.os.*
import android.text.InputFilter
import android.text.InputType
import android.text.TextUtils
import android.view.*
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowCompat
import androidx.core.view.WindowInsetsCompat
import java.time.Duration
```

```
import java.time.Instant

// Tela principal: abas Travessias, Mapa, Relatos e Bastidores. Recebe os dados do servidor e os avisos ao vivo
// pelo WebSocket; por garantia, busca tudo de novo a cada 30 segundos.
class MainActivity : AppCompatActivity() {
    private val prefs by lazy { getSharedPreferences("viaagora_private", MODE_PRIVATE) }
    private val estado = EstadoApp()
    private lateinit var ui: Ui
    private lateinit var painelMapa: PainelMapa
    private lateinit var textoStatus: TextView
    private lateinit var abas: List<TextView>
    private lateinit var listaTravessias: LinearLayout
    private lateinit var listaRelatos: LinearLayout
    private lateinit var paginas: List<View>
    private lateinit var bastidores: PaginaBastidores
    private lateinit var botaoNovoRelato: View
    private var remoto: RepositorioRemoto? = null
    private var dados = Panorama(catalogoTravessias, emptyList(), InfoFonte(null, false, null, null))
    private var aba = 0
    private var filtroTravessia: String? = null
    private var aoVivo = false
    private var erroConexao = false
    private var localizacaoPendente: (() -> Unit)? = null
    private var ouvinteLocalizacao: LocationListener? = null
    private val agendador = Handler(Looper.getMainLooper())
    // Busca de novo a cada 30 s: se o canal ao vivo cair (app em segundo plano, troca de rede), os relatos
ainda chegam.
    private val atualizacaoPeriodica = object : Runnable { override fun run() { recarregar();
agendador.postDelayed(this, 30_000) } }
    private var relatosConhecidos: Set<String>? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        ui = Ui(this)
        WindowCompat.setDecorFitsSystemWindows(window, false)
        window.statusBarColor = Color.TRANSPARENT
        window.navigationBarColor = Color.WHITE
        WindowCompat.getInsetsController(window, window.decorView).apply { isAppearanceLightStatusBars = true;
isAppearanceLightNavigationBars = true }
        val raiz = FrameLayout(this).apply { setBackgroundColor(Paleta.fundo) }
        ViewCompat.setOnApplyWindowInsetsListener(raiz) { view, insets ->
            val barras = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            view.setPadding(barras.left, barras.top, barras.right, barras.bottom); insets
        }
        setContentView(raiz)
        val tela = ui.coluna()
        raiz.addView(tela, FrameLayout.LayoutParams(FrameLayout.LayoutParams.MATCH_PARENT,
FrameLayout.LayoutParams.MATCH_PARENT))

        val cabecalho = ui.linha().apply { setPadding(dp(20), dp(18), dp(8), dp(12)) }
        val titulos = ui.coluna()
        titulos.addView(ui.text("Balsas agora", 28f, Paleta.tinta, true))
        textoStatus = ui.text("", 13f, Paleta.apagado).apply { setPadding(0, dp(6), 0, 0); setOnClickListener {
recarregar() } }
        titulos.addView(textoStatus)
        cabecalho.addView(titulos, LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
        cabecalho.addView(ImageView(this).apply { setImageDrawable(DesenhoIcone("sliders", Paleta.tinta));
setPadding(dp(12), dp(12), dp(12), dp(12))
            contentDescription = "Ajustes"; isClickable = true; setOnClickListener { abrirAjustes() } },
LinearLayout.LayoutParams(dp(48), dp(48)))
        tela.addView(cabecalho)

        val conteudo = FrameLayout(this)
        tela.addView(conteudo, LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, 0, 1f))
        listaTravessias = ui.coluna().apply { setPadding(dp(16), dp(4), dp(16), dp(96)) }
        listaRelatos = ui.coluna().apply { setPadding(dp(16), dp(4), dp(16), dp(96)) }
        painelMapa = PainelMapa(this, savedInstanceState, ::abrirMarcador, { lat, lng -> abrirNovoRelato(null,
lat, lng) }, ::localizar)
        bastidores = PaginaBastidores(this, ui)
        paginas = listOf(ScrollView(this).apply { addView(listaTravessias) }, painelMapa, ScrollView(this).apply
{ addView(listaRelatos) }, bastidores.view)
        paginas.forEach { conteudo.addView(it, FrameLayout.LayoutParams(FrameLayout.LayoutParams.MATCH_PARENT,
FrameLayout.LayoutParams.MATCH_PARENT)) }

        val navegacao = ui.linha().apply { setBackgroundColor(Color.WHITE) }
        abas = listOf("Travessias", "Mapa", "Relatos", "Bastidores").mapIndexed { indice, rotulo ->
            ui.text(rotulo, 13f, Paleta.apagado, true).apply { gravity = Gravity.CENTER; isClickable = true;
contentDescription = rotulo; setOnClickListener { aba = indice; atualizarTela() } }
                .also { navegacao.addView(it, LinearLayout.LayoutParams(0, dp(60), 1f)) }
        }
        tela.addView(View(this).apply { setBackgroundColor(Paleta.linha) },
LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, dp(1)))
        tela.addView(navegacao)

        botaoNovoRelato = ui.text("+", 30f, Color.WHITE).apply {
```

```
            gravity = Gravity.CENTER; background = ui.arredondado(Paleta.destaque, 30); elevation =
dp(6).toFloat()
            contentDescription = "Relatar fila ou problema"; isClickable = true
            setOnClickListener { if (aba == 1) { val p = painelMapa.centro(); abrirNovoRelato(null, p.first,
p.second) } else abrirNovoRelato(null) }
        }
        raiz.addView(botaoNovoRelato, FrameLayout.LayoutParams(dp(60), dp(60), Gravity.BOTTOM or
Gravity.END).apply { setMargins(0, 0, dp(20), dp(80)) })

        // Abre no servidor oficial; quem escolheu outro servidor nos Ajustes continua nele.
        val salvo = prefs.getString("last_server", "").orEmpty().ifBlank { SERVIDOR_OFICIAL }
        runCatching { ativar(salvo) }.onFailure { ativar(SERVIDOR_OFICIAL) }
    }

    private fun ativar(servidor: String) {
        remoto?.fechar(); remoto = null
        estado.selecionar(servidor)
        aoVivo = false; erroConexao = false; relatosConhecidos = null
        bastidores.fonte = { ok, falhou -> remoto?.sistema(ok, falhou) }
        bastidores.servidor = estado.servidor
        bastidores.reiniciar()
        val repositorio = RepositorioRemoto(prefs, estado.servidor); remoto = repositorio
        dados = repositorio.emCache() ?: Panorama(catalogoTravessias, emptyList(), InfoFonte(null, false, null,
null)); atualizarTela()
        val senha = estado.geracao
        repositorio.conectar({ if (estado.aceita(senha)) recarregar() }, { connected -> if
(estado.aceita(senha)) { aoVivo = connected; atualizarTela() } },
            { tipo, replica, rastreavel -> if (estado.aceita(senha)) bastidores.aoEventoAoVivo(tipo, replica,
rastreavel) })
        recarregar()
    }
    private fun recarregar() {
        val senha = estado.geracao
        remoto?.buscar({ panorama -> if (estado.aceita(senha)) { erroConexao = false; atualizar(panorama) } },
            { message -> if (estado.aceita(senha)) { erroConexao = true; atualizarTela(); mostrarAviso(message)
} })
    }
    private fun atualizar(panorama: Panorama) {
        dados = panorama
        // Relato novo de outra pessoa: avisa na hora (na primeira carga só memoriza o que já existia).
        val ids = panorama.relatos.map { it.id }.toSet()
        relatosConhecidos?.let { conhecidos ->
            panorama.relatos.firstOrNull { it.id !in conhecidos && it.situacao == "active" && it.autorId !=
autorId() }
                ?.let { mostrarAviso("Novo relato · ${tituloRelato(it)}") }
        }
        relatosConhecidos = ids
        atualizarTela()
    }

    // Uma única linha de status; vermelho/âmbar só quando há algo errado.
    private fun atualizarStatus() {
        val fonte = dados.fonte
        val (text, color) = when {
            erroConexao -> (if (fonte.ultimoSucessoEm != null) "Sem conexão · dados de
${tempoRelativo(fonte.ultimoSucessoEm)} · toque para tentar"
                else "Sem conexão · toque para tentar de novo") to Paleta.perigo
            fonte.ultimoSucessoEm == null -> "Aguardando o painel oficial" to Paleta.apagado
            !fonte.disponivel -> "Painel oficial fora do ar · último dado
${tempoRelativo(fonte.ultimoSucessoEm)}" to Paleta.alerta
            aoVivo -> "● Ao vivo · atualizado ${tempoRelativo(fonte.ultimoSucessoEm)}" to Paleta.destaque
            else -> "Atualizado ${tempoRelativo(fonte.ultimoSucessoEm)}" to Paleta.apagado
        }
        textoStatus.text = text; textoStatus.setTextColor(color)
    }
    private fun atualizarTela() {
        atualizarStatus()
        abas.forEachIndexed { indice, view -> view.setTextColor(if (indice == aba) Paleta.destaque else
Paleta.apagado) }
        paginas.forEachIndexed { indice, view -> view.visibility = if (indice == aba) View.VISIBLE else
View.GONE }
        botaoNovoRelato.visibility = if (aba == 3) View.GONE else View.VISIBLE
        if (aba == 3) bastidores.show() else bastidores.hide()
        listaTravessias.removeAllViews()
        dados.travessias.forEach { listaTravessias.addView(cartaoTravessia(it));
listaTravessias.addView(ui.espaco(12)) }
        val ativos = dados.relatos.filter { it.situacao == "active" }
        painelMapa.atualizar(dados.travessias.flatMap { c -> listOf("A", "B").map { lado ->
            val t = c.terminal(lado); MarcadorMapa("terminal:${c.id}:$lado", t.latitude, t.longitude,
nivelEspera(espera(c, lado), c.situacao?.operando ?: true).color, true) } } +
            ativos.map { MarcadorMapa("report:${it.id}", it.latitude, it.longitude,
infoCategoria(it.categoria).color, false) },
            dados.travessias.map { RotaMapa(it.rota, if (it.parada) "#DC2626" else "#0B7A8A") })
        mostrarRelatos()
    }
```

```
    private fun espera(travessia: Travessia, lado: String) = travessia.situacao?.let { if (lado == "A") it.esperaA else it.esperaB }
    private fun corDoNivel(c: Travessia, lado: String) = Color.parseColor(nivelEspera(espera(c, lado), c.situacao?.operando ?: true).color)
    // Com a travessia parada, mostrar "0 min" engana: aparece "Parada" no lugar dos minutos.
    private fun valorDoLado(c: Travessia, lado: String, size: Int = 32): CharSequence =
        if (c.parada) ui.grande("Parada", Paleta.perigo, size * 3 / 4) else ui.minutos(espera(c, lado), corDoNivel(c, lado), size)
    private fun relatosRecentes(c: Travessia) = dados.relatos.filter { it.travessiaId == c.id && it.situacao == "active" }
    private fun textoRelatosParada(c: Travessia) = "⚠ ${c.relatosParada} relato${if (c.relatosParada > 1) "s" else ""} de travessia parada"

    private fun cartaoTravessia(c: Travessia): View {
        val s = c.situacao
        val cartao = ui.coluna().apply {
            setPadding(dp(20), dp(18), dp(20), dp(18)); background = ui.arredondado(Color.WHITE, 20)
            isClickable = true; isFocusable = true; setOnClickListener { detalhesTravessia(c) }
        }
        val titulo = ui.linha()
        titulo.addView(ui.text(c.nome, 17f, Paleta.tinta, true), LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
        if (s != null) titulo.addView(ui.text(if (s.operando) "🚢 ${s.embarcacoes}" else "🚢 0", 13f, if (s.operando) Paleta.apagado else Paleta.perigo))
        cartao.addView(titulo); cartao.addView(ui.espaco(14))
        val lados = ui.linha().apply { gravity = Gravity.TOP }
        listOf("A", "B").forEachIndexed { indice, lado ->
            if (indice > 0) lados.addView(View(this).apply { setBackgroundColor(Paleta.linha) }, LinearLayout.LayoutParams(dp(1), dp(44)).apply { marginEnd = dp(16) })
            val caixa = ui.coluna()
            caixa.addView(ui.text(c.terminal(lado).nome, 13f, Paleta.apagado).apply { maxLines = 1; ellipsize = TextUtils.TruncateAt.END })
            caixa.addView(ui.espaco(4))
            caixa.addView(ui.text(valorDoLado(c, lado), 15f, Paleta.tinta, true))
            (if (lado == "A") c.comunidadeA else c.comunidadeB)?.let { caixa.addView(ui.text("~${it.minutosEspera} min pela comunidade", 12f, Paleta.apagado).apply { setPadding(0, dp(4), 0, 0) }) }
            lados.addView(caixa, LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
        }
        cartao.addView(lados)
        val (message, color) = when {
            s == null -> "Sem leitura do painel ainda" to Paleta.apagado
            c.parada -> (s.aviso?.takeIf { it.isNotBlank() } ?: "Sem embarcação em operação") to Paleta.perigo
            s.atrasado -> "Dado atrasado · ${tempoRelativo(s.coletadoEm)}" to Paleta.alerta
            c.relatosParada > 0 -> textoRelatosParada(c) to Paleta.perigo
            !s.aviso.isNullOrBlank() -> "⚠ ${s.aviso}" to Paleta.alerta
            else -> null to Paleta.apagado
        }
        if (message != null) cartao.addView(ui.text(message, 13f, color).apply { setPadding(0, dp(12), 0, 0); maxLines = 2; ellipsize = TextUtils.TruncateAt.END })
        relatosRecentes(c).takeIf { it.isNotEmpty() }?.let { recentes ->
            val last = recentes.first()
            cartao.addView(ui.text("💬 ${recentes.size} relato${if (recentes.size > 1) "s" else ""} · ${infoCategoria(last.categoria).rotulo} ${tempoRelativo(last.criadoEm)}",
                13f, Paleta.destaque).apply { setPadding(0, dp(10), 0, 0); maxLines = 1; ellipsize = TextUtils.TruncateAt.END })
        }
        cartao.contentDescription = if (c.parada) "${c.nome}: travessia parada. ${s?.aviso.orEmpty()}"
            else "${c.nome}. ${c.pontoA.nome}: ${espera(c, "A") ?: "sem dado"} minutos. ${c.pontoB.nome}: ${espera(c, "B") ?: "sem dado"} minutos."
        return cartao
    }
    private fun detalhesTravessia(c: Travessia, lado: String? = null) {
        val s = c.situacao
        val corpo = ui.coluna()
        corpo.addView(ui.text(rotuloTipo(c.tipo), 13f, Paleta.apagado))
        corpo.addView(ui.espaco(18))
        val lados = ui.linha()
        listOf("A", "B").forEach { sd ->
            val caixa = ui.coluna().apply { setPadding(dp(14), dp(12), dp(14), dp(12)); background = ui.arredondado(if (sd == lado) Paleta.suave else Paleta.fundo, 14) }
            caixa.addView(ui.text(c.terminal(sd).nome, 13f, Paleta.apagado).apply { maxLines = 1; ellipsize = TextUtils.TruncateAt.END })
            caixa.addView(ui.espaco(4))
            caixa.addView(ui.text(valorDoLado(c, sd, 28), 15f, Paleta.tinta, true))
            if (!c.parada) caixa.addView(ui.text(nivelEspera(espera(c, sd), s?.operando ?: true).rotulo, 12f, corDoNivel(c, sd)).apply { setPadding(0, dp(4), 0, 0) })
            (if (sd == "A") c.comunidadeA else c.comunidadeB)?.let { caixa.addView(ui.text("~${it.minutosEspera} min · ${it.relatos} relato${if (it.relatos > 1) "s" else ""}", 12f, Paleta.apagado).apply { setPadding(0, dp(4), 0, 0) }) }
            lados.addView(caixa, LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f).apply { if (sd == "B") marginStart = dp(10) })
        }
        corpo.addView(lados)
```

```
224          if (s != null) {
225              corpo.addView(ui.espaco(16))
226              listOfNotNull(
227                  if (c.parada) "⛔ Travessia parada pelo painel oficial" else null,
228                  if (c.relatosParada > 0) textoRelatosParada(c) else null,
229                  if (s.operando) "${s.embarcacoes} ${if (s.embarcacoes == 1) "embarcação" else "embarcações"} em
operação" else "Sem embarcação em operação",
230                  s.climaA?.takeIf { it.isNotBlank() }?.let { "Tempo: $it" },
231                  s.aviso?.takeIf { it.isNotBlank() }?.let { "⚠ $it" },
232                  "Painel oficial · ${tempoRelativo(s.coletadoEm)}",
233              ).forEach { linha -> corpo.addView(ui.text(linha, 14f, when { linha.startsWith("⛔") ||
linha.contains("travessia parada") -> Paleta.perigo; linha.startsWith("⚠") -> Paleta.alerta; else -> Paleta.tinta
}).apply { setPadding(0, dp(5), 0, dp(5)) }) }
234          }
235          val recentes = relatosRecentes(c)
236          lateinit var painel: android.app.Dialog
237          if (recentes.isNotEmpty()) {
238              corpo.addView(ui.rotulo("Relatos recentes"))
239              recentes.take(3).forEach { item -> corpo.addView(cartaoRelato(item).apply { setOnClickListener {
painel.dismiss(); detalhesRelato(item) } }); corpo.addView(ui.espaco(6)) }
240          }
241          corpo.addView(ui.espaco(20))
242          painel = ui.painel(c.nome, corpo)
243          val acoes = ui.linha()
244          listOf("Melhor horário" to { painel.dismiss(); historico(c) }, "Relatar" to { painel.dismiss();
abrirNovoRelato(c.id, ladoInicial = lado ?: "A") })
245              .forEachIndexed { indice, (rotulo, action) ->
246                  acoes.addView(ui.botao(rotulo, indice == 1, action), LinearLayout.LayoutParams(0, dp(48),
1f).apply { if (indice > 0) marginStart = dp(8) })
247              }
248          corpo.addView(acoes)
249          painel.show()
250      }
251
252      private fun mostrarRelatos() {
253          listaRelatos.removeAllViews()
254          val filtros = listOf<String?>(null) + dados.travessias.map { it.id }
255          val rotulos = listOf("Todas") + dados.travessias.map { it.nome }
256          listaRelatos.addView(Escolha(ui, rotulos, filtros.indexOf(filtroTravessia).coerceAtLeast(0)) {
filtroTravessia = filtros[it]; atualizarTela() }.view)
257          listaRelatos.addView(ui.espaco(14))
258          val visivel = filtrarRelatos(dados.relatos, filtroTravessia, false)
259          if (visivel.isEmpty()) listaRelatos.addView(ui.text("Nenhum relato recente.\nEstá na fila? Toque em + e
conte quanto está esperando.", 15f, Paleta.apagado).apply {
260              gravity = Gravity.CENTER; setPadding(dp(24), dp(48), dp(24), dp(24)) })
261          visivel.forEach { listaRelatos.addView(cartaoRelato(it)); listaRelatos.addView(ui.espaco(10)) }
262      }
263      private fun travessiaPorId(id: String) = dados.travessias.firstOrNull { it.id == id } ?:
catalogoTravessias.first { it.id == id }
264      // Onde foi o relato: a travessia e o lado da fila (ex.: "Santos ⇄ Guarujá · lado Santos").
265      private fun localDoRelato(item: Relato) = travessiaPorId(item.travessiaId).let { "${it.nome} · lado
${it.terminal(item.lado).nome}" }
266      private fun tituloRelato(item: Relato) = "${infoCategoria(item.categoria).rotulo} · ${localDoRelato(item)}"
267      private fun cartaoRelato(item: Relato): View {
268          val info = infoCategoria(item.categoria); val color = Color.parseColor(info.color)
269          val cartao = ui.linha().apply { setPadding(dp(16), dp(14), dp(16), dp(14)); background =
ui.arredondado(Color.WHITE, 18); isClickable = true; isFocusable = true; setOnClickListener { detalhesRelato(item) }
}
270          cartao.addView(ui.text(info.simbolo, 18f, color, true).apply { gravity = Gravity.CENTER; background =
ui.arredondado(Paleta.fundo, 20) },
271              LinearLayout.LayoutParams(dp(40), dp(40)).apply { marginEnd = dp(14) })
272          val text = ui.coluna()
273          val topo = ui.linha()
274          topo.addView(ui.text(info.rotulo, 15f, Paleta.tinta, true), LinearLayout.LayoutParams(0,
LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
275          if (item.autorId == autorId()) topo.addView(ui.text("Seu relato", 11f, Paleta.destaque, true).apply {
276              background = ui.arredondado(Paleta.suave, 10); setPadding(dp(8), dp(3), dp(8), dp(3)) })
277          text.addView(topo)
278          text.addView(ui.text(localDoRelato(item), 14f, Paleta.tinta).apply { setPadding(0, dp(4), 0, 0) })
279          text.addView(ui.text(listOfNotNull(item.minutosEspera?.let { "Espera de $it min" }, "enviado
${tempoRelativo(item.criadoEm)}").joinToString(" · ")
280              .replaceFirstChar { it.uppercase() }, 13f, Paleta.apagado).apply { setPadding(0, dp(4), 0, 0) })
281          cartao.addView(text, LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
282          cartao.contentDescription = tituloRelato(item)
283          return cartao
284      }
285      private fun detalhesRelato(item: Relato) {
286          val corpo = ui.coluna()
287          val travessia = travessiaPorId(item.travessiaId)
288          listOf("Travessia" to travessia.nome, "Lado da fila" to travessia.terminal(item.lado).nome,
289              "Espera informada" to (item.minutosEspera?.let { "$it min" } ?: "não informada"),
290              (if (item.situacao == "active") "Enviado" else "Encerrado") to tempoRelativo(item.criadoEm),
291          ).forEach { (rotulo, valor) ->
292              val linha = ui.linha().apply { setPadding(0, dp(7), 0, dp(7)) }
```

```
            linha.addView(ui.text(rotulo, 14f, Paleta.apagado), LinearLayout.LayoutParams(0,
LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
            linha.addView(ui.text(valor, 14f, Paleta.tinta, true))
            corpo.addView(linha)
        }
        item.descricao.takeIf { it.isNotBlank() }?.let {
            corpo.addView(ui.text("“$it”", 15f, Paleta.tinta).apply { setPadding(dp(14), dp(12), dp(14),
dp(12)); background = ui.arredondado(Paleta.suave, 12)
                layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT,
LinearLayout.LayoutParams.WRAP_CONTENT).apply { topMargin = dp(8) } })
        }
        corpo.addView(ui.text("Informação colaborativa, sem confirmação oficial.", 13f, Paleta.apagado).apply {
setPadding(0, dp(14), 0, 0) })
        corpo.addView(ui.espaco(18))
        val painel = ui.painel(infoCategoria(item.categoria).rotulo, corpo)
        val acoes = ui.linha()
        acoes.addView(ui.botao("Ver no mapa") { painel.dismiss(); aba = 1; atualizarTela();
painelMapa.focar(item.latitude, item.longitude) }, LinearLayout.LayoutParams(0, dp(48), 1f))
        if (item.situacao == "active" && item.autorId == autorId()) acoes.addView(ui.botao("Encerrar", true) {
            painel.dismiss()
            val senha = estado.geracao
            remoto?.encerrar(item.id, { if (estado.aceita(senha)) { recarregar(); mostrarAviso("Relato
encerrado.") } }, ::falha)
        }, LinearLayout.LayoutParams(0, dp(48), 1f).apply { marginStart = dp(8) })
        corpo.addView(acoes)
        painel.show()
    }
    private fun abrirMarcador(id: String) {
        val partes = id.split(":")
        if (partes[0] == "report") dados.relatos.firstOrNull { it.id == partes[1] }?.let(::detalhesRelato)
        else detalhesTravessia(travessiaPorId(partes[1]), partes[2])
    }

    private fun historico(c: Travessia) {
        val show = { h: Historico ->
            val corpo = ui.coluna()
            if (h.porHora.isEmpty()) corpo.addView(ui.text("Ainda não há leituras suficientes. Volte mais
tarde.", 15f, Paleta.apagado).apply { setPadding(0, dp(12), 0, dp(12)) })
            else {
                corpo.addView(ui.text("Espera média por hora, últimos 7 dias", 13f, Paleta.apagado))
                corpo.addView(ui.espaco(16)); corpo.addView(GraficoHistorico(this, h));
corpo.addView(ui.espaco(16))
                listOf("A" to h.melhorA, "B" to h.melhorB).forEach { (lado, hora) ->
                    if (hora == null) return@forEach
                    val media = h.porHora.first { it.hora == hora }.let { if (lado == "A") it.esperaA else
it.esperaB }
                    val linha = ui.linha().apply { setPadding(0, dp(6), 0, dp(6)) }
                    linha.addView(ui.text("■", 14f, if (lado == "A") Paleta.marinho else Paleta.destaque).apply
{ setPadding(0, 0, dp(10), 0) })
                    linha.addView(ui.text("Saindo de ${c.terminal(lado).nome}", 15f),
LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
                    linha.addView(ui.text("${hora}h · ~$media min", 15f,
Color.parseColor(NivelEspera.BAIXA.color), true))
                    corpo.addView(linha)
                }
            }
            ui.painel("Melhor horário", corpo).show()
        }
        val senha = estado.geracao
        remoto?.historico(c.id, { if (estado.aceita(senha)) show(it) }, ::falha)
    }
    private fun abrirNovoRelato(travessiaId: String?, lat: Double? = null, lng: Double? = null, ladoInicial:
String = "A") {
        val corpo = ui.coluna()
        var travessia = catalogoTravessias.firstOrNull { it.id == travessiaId } ?: catalogoTravessias.first()
        lateinit var lado: Escolha
        corpo.addView(ui.rotulo("Travessia"))
        corpo.addView(Escolha(ui, catalogoTravessias.map { it.nome }, catalogoTravessias.indexOf(travessia)) {
            travessia = catalogoTravessias[it]; lado.trocarRotulos(listOf("Em ${travessia.pontoA.nome}", "Em
${travessia.pontoB.nome}"))
        }.view)
        corpo.addView(ui.rotulo("Onde você está"))
        lado = Escolha(ui, listOf("Em ${travessia.pontoA.nome}", "Em ${travessia.pontoB.nome}"), if (ladoInicial
== "B") 1 else 0)
        corpo.addView(lado.view)
        corpo.addView(ui.rotulo("O que está acontecendo"))
        val categoria = Escolha(ui, categorias.map { it.rotulo })
        corpo.addView(categoria.view)
        corpo.addView(ui.rotulo("Detalhes (opcional)"))
        val campoEspera = ui.entrada("Minutos de espera").apply { inputType = InputType.TYPE_CLASS_NUMBER;
filters = arrayOf(InputFilter.LengthFilter(3)) }
        corpo.addView(campoEspera); corpo.addView(ui.espaco(8))
        val descricao = ui.entrada("Ex.: a fila chega até o canal 6").apply {
            inputType = InputType.TYPE_CLASS_TEXT or InputType.TYPE_TEXT_FLAG_CAP_SENTENCES; filters =
arrayOf(InputFilter.LengthFilter(400)) }
```

```
361          corpo.addView(descricao)
362          if (lat != null) corpo.addView(ui.text("📍 Ponto escolhido no mapa", 13f, Paleta.apagado).apply {
setPadding(0, dp(12), 0, 0) })
363          corpo.addView(ui.espaco(24))
364          val painel = ui.painel("Novo relato", corpo)
365          corpo.addView(ui.botao("Publicar", true) {
366              val letraLado = if (lado.selecionada == 1) "B" else "A"
367              val terminal = travessia.terminal(letraLado)
368              val textoEspera = campoEspera.text.toString().trim()
369              val espera = if (textoEspera.isEmpty()) null else textoEspera.toIntOrNull() ?: -1
370              val ponto = if (lat != null && lng != null) lat to lng else terminal.latitude to terminal.longitude
371              val cat = categorias[categoria.selecionada].key
372              val error = validarRelato(travessia.id, letraLado, cat, espera, descricao.text.toString(),
ponto.first, ponto.second)
373              if (error != null) { mostrarAviso(error); return@botao }
374              val entrada = NovoRelato(travessia.id, letraLado, cat, espera, descricao.text.toString(),
ponto.first, ponto.second)
375              val senha = estado.geracao
376              remoto?.criar(entrada, { if (estado.aceita(senha)) { painel.dismiss(); aba = 2; recarregar();
mostrarAviso("Relato publicado.") } },
377                  { message -> if (estado.aceita(senha)) mostrarAviso(message) })
378          }, LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, dp(52)))
379          painel.show()
380      }
381      // Ajustes: o normal é o servidor oficial. "Outro servidor" é para quem roda o sistema no próprio
computador.
382      private fun abrirAjustes() {
383          val corpo = ui.coluna()
384          corpo.addView(ui.text("De onde o app busca os dados", 13f, Paleta.apagado))
385          corpo.addView(ui.espaco(14))
386          val atual = prefs.getString("last_server", SERVIDOR_OFICIAL).orEmpty()
387          var personalizado = atual != SERVIDOR_OFICIAL
388          val endereco = ui.entrada("http://192.168.0.10:8080", if (personalizado) atual else "").apply {
389              inputType = InputType.TYPE_CLASS_TEXT or InputType.TYPE_TEXT_VARIATION_URI; isSingleLine = true }
390          val error = ui.text("", 13f, Paleta.perigo).apply { setPadding(dp(4), dp(8), 0, 0); visibility =
View.GONE }
391          val opcoes = listOf(
392              Triple("servers", "Servidor oficial", "sedda.com.br · dados reais e relatos de todos"),
393              Triple("server", "Outro servidor", "O sistema rodando no seu computador (Rodar local)"),
394          ).mapIndexed { indice, (icone, titulo, detalhe) ->
395              ui.linha().apply {
396                  setPadding(dp(14), dp(14), dp(14), dp(14)); isClickable = true; contentDescription = titulo
397                  addView(ImageView(this@MainActivity).apply { setImageDrawable(DesenhoIcone(icone,
Paleta.destaque)) }, LinearLayout.LayoutParams(dp(24), dp(24)).apply { marginEnd = dp(14) })
398                  addView(ui.coluna().apply { addView(ui.text(titulo, 15f, Paleta.tinta, true));
addView(ui.text(detalhe, 13f, Paleta.apagado).apply { setPadding(0, dp(3), 0, 0) }) },
399                      LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
400              }.also { linha -> corpo.addView(linha,
LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, LinearLayout.LayoutParams.WRAP_CONTENT).apply {
bottomMargin = dp(8) }) }
401          }
402          fun pintar() {
403              opcoes.forEachIndexed { i, linha -> val ativo = (i == 1) == personalizado; linha.background =
ui.arredondado(if (ativo) Paleta.suave else Color.WHITE, 14, if (ativo) Paleta.destaque else Paleta.linha) }
404              endereco.visibility = if (personalizado) View.VISIBLE else View.GONE; error.visibility = View.GONE
405          }
406          opcoes.forEachIndexed { i, linha -> linha.setOnClickListener { personalizado = i == 1; pintar() } }
407          corpo.addView(endereco); corpo.addView(error); pintar()
408          corpo.addView(ui.espaco(12))
409          corpo.addView(ui.text("Dados: painel das Travessias Litorâneas (SEMIL-SP). Relatos são colaborativos.
Mapa © OpenStreetMap.", 12f, Paleta.apagado))
410          corpo.addView(ui.espaco(20))
411          val painel = ui.painel("Ajustes", corpo)
412          corpo.addView(ui.botao("Aplicar", true) {
413              val url = if (personalizado) runCatching { normalizarServidor(endereco.text.toString()) }
414                  .getOrElse { error.text = it.message; error.visibility = View.VISIBLE; return@botao } else
SERVIDOR_OFICIAL
415              prefs.edit().putString("last_server", url).apply()
416              painel.dismiss(); ativar(url)
417          }, LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, dp(52)))
418          painel.show()
419      }
420      private fun autorId() = remoto?.autorId.orEmpty()
421      private fun falha(message: String) { erroConexao = true; atualizarTela(); mostrarAviso(message) }
422      private fun mostrarAviso(valor: String) { Toast.makeText(this, valor, Toast.LENGTH_LONG).show() }
423      private fun tempoRelativo(valor: String?): String = runCatching {
424          val minutos = Duration.between(Instant.parse(valor), Instant.now()).toMinutes().coerceAtLeast(0)
425          if (minutos < 1) "agora" else if (minutos < 60) "há $minutos min" else if (minutos < 1440) "há ${minutos
/ 60} h" else "há ${minutos / 1440} dias"
426      }.getOrDefault("—")
427
428      @Suppress("MissingPermission", "DEPRECATION")
429      private fun localizar() {
430          if (checkSelfPermission(Manifest.permission.ACCESS_COARSE_LOCATION) != PackageManager.PERMISSION_GRANTED
&& checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
```

```
431              localizacaoPendente = { localizar() };
requestPermissions(arrayOf(Manifest.permission.ACCESS_COARSE_LOCATION, Manifest.permission.ACCESS_FINE_LOCATION),
41); return
432          }
433          val gerenciador = getSystemService(LOCATION_SERVICE) as LocationManager
434          val provedores = listOf(LocationManager.NETWORK_PROVIDER, LocationManager.GPS_PROVIDER).filter {
runCatching { gerenciador.isProviderEnabled(it) }.getOrDefault(false) }
435          val conhecidos = provedores.mapNotNull { runCatching { gerenciador.getLastKnownLocation(it)
}.getOrNull() }
436              .filter { localizacaoRecente((SystemClock.elapsedRealtimeNanos() - it.elapsedRealtimeNanos) /
1_000_000L) }.maxByOrNull { it.time }
437          if (conhecidos != null) { painelMapa.focar(conhecidos.latitude, conhecidos.longitude); return }
438          if (provedores.isEmpty()) { mostrarAviso("Ative a localização ou escolha um ponto no mapa."); return }
439          mostrarAviso("Buscando sua posição…")
440          ouvinteLocalizacao?.let { gerenciador.removeUpdates(it) }
441          val ouvinte = object : LocationListener {
442              override fun onLocationChanged(localizacao: Location) { gerenciador.removeUpdates(this);
ouvinteLocalizacao = null; painelMapa.focar(localizacao.latitude, localizacao.longitude) }
443              override fun onProviderEnabled(provider: String) {}
444              override fun onProviderDisabled(provider: String) {}
445              @Deprecated("Deprecated in Android") override fun onStatusChanged(provider: String?, situacao: Int,
extras: Bundle?) {}
446          }
447          ouvinteLocalizacao = ouvinte
448          runCatching { gerenciador.requestLocationUpdates(provedores.first(), 0L, 0f, ouvinte,
Looper.getMainLooper()) }.onFailure { mostrarAviso("Não foi possível obter a posição.") }
449          agendador.postDelayed({ if (ouvinteLocalizacao === ouvinte) { gerenciador.removeUpdates(ouvinte);
ouvinteLocalizacao = null; mostrarAviso("Posição indisponível.") } }, 10000)
450      }
451      override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<out String>, grantResults:
IntArray) {
452          super.onRequestPermissionsResult(requestCode, permissions, grantResults)
453          if (requestCode == 41) { if (grantResults.any { it == PackageManager.PERMISSION_GRANTED })
localizacaoPendente?.invoke() else mostrarAviso("Sem problema: escolha o ponto no mapa."); localizacaoPendente = null
}
454      }
455      override fun onStart() { super.onStart(); painelMapa.start(); agendador.postDelayed(atualizacaoPeriodica,
30_000); if (aba == 3) bastidores.show() }
456      override fun onResume() { super.onResume(); painelMapa.resume(); recarregar() }
457      override fun onPause() { painelMapa.pause(); super.onPause() }
458      override fun onStop() { agendador.removeCallbacks(atualizacaoPeriodica); bastidores.hide();
painelMapa.stop(); super.onStop() }
459      override fun onLowMemory() { super.onLowMemory(); painelMapa.lowMemory() }
460      override fun onSaveInstanceState(outState: Bundle) { super.onSaveInstanceState(outState);
painelMapa.save(outState) }
461      override fun onDestroy() { remoto?.fechar(); agendador.removeCallbacksAndMessages(null);
ouvinteLocalizacao?.let { runCatching { (getSystemService(LOCATION_SERVICE) as LocationManager).removeUpdates(it) }
}; painelMapa.destroy(); super.onDestroy() }
462  }
```

android/app/src/main/java/br/com/viaagora/RepositorioRemoto.kt

```
 1  package br.com.viaagora
 2
 3  import android.content.SharedPreferences
 4  import android.os.Handler
 5  import android.os.Looper
 6  import okhttp3.*
 7  import okhttp3.MediaType.Companion.toMediaType
 8  import okhttp3.RequestBody.Companion.toRequestBody
 9  import org.json.JSONObject
10  import java.security.MessageDigest
11  import java.util.UUID
12  import java.util.concurrent.Executors
13  import java.util.concurrent.TimeUnit
14
15  // Comunicação com o servidor: HTTP para buscar e enviar dados e WebSocket para receber os avisos ao vivo.
16  // As chamadas rodam fora da linha principal da tela e a última resposta fica guardada para abrir o app sem
internet.
17  class RepositorioRemoto(private val prefs: SharedPreferences, val servidor: String) {
18      private val cliente = OkHttpClient.Builder().connectTimeout(6, TimeUnit.SECONDS).readTimeout(10,
TimeUnit.SECONDS)
19          .callTimeout(15, TimeUnit.SECONDS).pingInterval(20, TimeUnit.SECONDS).build()
20      private val io = Executors.newSingleThreadExecutor()
21      private val principal = Handler(Looper.getMainLooper())
22      private var socket: WebSocket? = null
23      @Volatile private var fechado = false
24      val autorId: String get() = prefs.getString("participant:$servidor", "") ?: ""
25      fun emCache(): Panorama? = runCatching {
26          val (travessias, fonte) = lerTravessias(prefs.getString("crossings:$servidor", null) ?: return null)
27          Panorama(travessias, lerRelatos(prefs.getString("reports:$servidor", "[]") ?: "[]"), fonte)
28      }.getOrNull()
29      private fun token() = prefs.getString("token:$servidor", "") ?: ""
30      private fun requisitar(method: String, caminho: String, dados: JSONObject? = null, autenticar: Boolean =
false, key: String? = null): JSONObject {
```

```
        val body = dados?.toString()?.toRequestBody("application/json; charset=utf-8".toMediaType())
        val construtor = Request.Builder().url(servidor + caminho).header("User-Agent", "ViaAgora-Android/1.0")
        if (autenticar) construtor.header("Authorization", "Bearer ${token()}")
        if (key != null) construtor.header("Idempotency-Key", key)
        construtor.method(method, if (method in listOf("POST", "PATCH")) body ?:
"{}".toRequestBody("application/json".toMediaType()) else null)
        return cliente.newCall(construtor.build()).execute().use { resposta ->
            val text = resposta.body?.string().orEmpty()
            val json = runCatching { JSONObject(text) }.getOrDefault(JSONObject())
            if (!resposta.isSuccessful) {
                if (resposta.code == 401)
prefs.edit().remove("token:$servidor").remove("participant:$servidor").apply()
                error(json.optString("message", "Servidor respondeu com erro ${resposta.code}."))
            }
            json
        }
    }
    private fun garantirParticipante() {
        if (token().isNotEmpty()) return
        val resultado = requisitar("POST", "/v1/participants", JSONObject())
        prefs.edit().putString("participant:$servidor",
resultado.getString("participantId")).putString("token:$servidor", resultado.getString("token")).commit()
    }
    private fun executarEmSegundoPlano(aoErro: (String) -> Unit, acao: () -> Unit) {
        if (fechado) return
        runCatching { io.execute { try { acao() } catch (e: Exception) {
            val message = e.message?.takeIf { it.length < 180 } ?: "Não foi possível alcançar o servidor.
Confira a conexão."
            principal.post { if (!fechado) aoErro(message) }
        } } }
    }
    // Busca as travessias e os relatos e guarda uma cópia local.
    fun buscar(aoSucesso: (Panorama) -> Unit, aoErro: (String) -> Unit) = executarEmSegundoPlano(aoErro) {
        val textoTravessias = requisitar("GET", "/v1/crossings").toString()
        val relatos = lerRelatos(requisitar("GET", "/v1/reports?limit=500").getJSONArray("reports").toString())
        val (travessias, fonte) = lerTravessias(textoTravessias)
        prefs.edit().putString("crossings:$servidor", textoTravessias).putString("reports:$servidor",
codificarRelatos(relatos)).apply()
        principal.post { if (!fechado) aoSucesso(Panorama(travessias, relatos, fonte)) }
    }
    fun historico(travessiaId: String, aoSucesso: (Historico) -> Unit, aoErro: (String) -> Unit) =
executarEmSegundoPlano(aoErro) {
        val historico = lerHistorico(requisitar("GET", "/v1/crossings/$travessiaId/history?days=7").toString())
        principal.post { if (!fechado) aoSucesso(historico) }
    }
    fun sistema(aoSucesso: (EstadoSistema) -> Unit, aoErro: (String) -> Unit) = executarEmSegundoPlano(aoErro) {
        val estado = lerSistema(requisitar("GET", "/v1/system").toString())
        principal.post { if (!fechado) aoSucesso(estado) }
    }
    // Envia um relato. A chave de idempotência fica salva até o servidor confirmar: se a rede cair e o app
    // tentar de novo, o servidor reconhece o mesmo envio e não cria um relato duplicado.
    fun criar(entrada: NovoRelato, aoSucesso: (Relato) -> Unit, aoErro: (String) -> Unit) =
executarEmSegundoPlano(aoErro) {
        garantirParticipante()
        val body = entrada.json()
        val impressaoDigital = MessageDigest.getInstance("SHA-
256").digest(body.toString().toByteArray()).joinToString("") { "%02x".format(it) }
        val chavePendente = "pending:$servidor:$impressaoDigital"
        val key = prefs.getString(chavePendente, null) ?: UUID.randomUUID().toString().also {
prefs.edit().putString(chavePendente, it).commit() }
        val item = lerRelato(requisitar("POST", "/v1/reports", body, true, key).getJSONObject("report"))
        prefs.edit().remove(chavePendente).apply()
        principal.post { if (!fechado) aoSucesso(item) }
    }
    fun encerrar(id: String, aoSucesso: (Relato) -> Unit, aoErro: (String) -> Unit) =
executarEmSegundoPlano(aoErro) {
        garantirParticipante()
        val item = lerRelato(requisitar("PATCH", "/v1/reports/$id/resolve", JSONObject(),
true).getJSONObject("report"))
        principal.post { if (!fechado) aoSucesso(item) }
    }
    // aoEvento recebe cada mensagem do WebSocket (tipo, réplica que entregou, se é um evento rastreável) para a
aba Bastidores.
    fun conectar(aoMudar: () -> Unit, aoMudarConexao: (Boolean) -> Unit, aoEvento: (String, String?, Boolean) ->
Unit = { _, _, _ -> }) {
        if (fechado) return
        val endereco = servidor.replaceFirst("https://", "wss://").replaceFirst("http://", "ws://") +
"/v1/events"
        socket = cliente.newWebSocket(Request.Builder().url(endereco).build(), object: WebSocketListener() {
            override fun onMessage(webSocket: WebSocket, text: String) {
                val evento = runCatching { JSONObject(text) }.getOrNull() ?: return
                principal.post { if (!fechado) {
                    aoEvento(evento.optString("type"), evento.optString(if (evento.has("replica")) "replica"
else "via").takeIf { it.isNotBlank() }, evento.has("eventId"))
                    when (evento.optString("type")) {
```

```
                        "ready", "resync" -> { aoMudarConexao(evento.optBoolean("connected", true)); aoMudar() }
                        "broker.status" -> aoMudarConexao(evento.optBoolean("connected", false))
                        else -> aoMudar()
                    }
                } }
            }
            override fun onFailure(webSocket: WebSocket, t: Throwable, resposta: Response?) { reconectar() }
            override fun onClosed(webSocket: WebSocket, code: Int, reason: String) { reconectar() }
            private fun reconectar() { principal.post { if (!fechado) { aoMudarConexao(false);
principal.postDelayed({ conectar(aoMudar, aoMudarConexao, aoEvento) }, 2500) } } }
        })
    }
    fun fechar() { fechado = true; principal.removeCallbacksAndMessages(null); socket?.cancel();
cliente.dispatcher.cancelAll(); io.shutdownNow() }
}
```

android/app/src/main/java/br/com/viaagora/EstadoApp.kt

```
package br.com.viaagora

import java.net.URI

// Qual servidor o app usa: o oficial (VPS) ou o do computador de quem roda o projeto.
// Confere e padroniza o endereço digitado em Ajustes → Outro servidor.
fun normalizarServidor(valor: String): String {
    val uri = URI(valor.trim())
    require(uri.scheme?.lowercase() in listOf("http", "https") && !uri.host.isNullOrBlank()) { "Use um endereço
completo, começando com http:// ou https://." }
    require(uri.userInfo == null && uri.query == null && uri.fragment == null && uri.port in -1..65535) { "O
endereço não deve conter senha, parâmetros ou fragmentos." }
    return URI(uri.scheme.lowercase(), null, uri.host.lowercase(), uri.port, uri.path.orEmpty().trimEnd('/'),
null, null).toString()
}
// Servidor oficial da APS: o app abre conectado a ele, sem o usuário digitar endereço.
const val SERVIDOR_OFICIAL = "https://sedda.com.br"

// Cada troca de servidor gera uma nova "geração". Respostas que chegam de um servidor anterior são descartadas.
class EstadoApp {
    var geracao: Int = 0
        private set
    var servidor: String = ""
        private set
    fun selecionar(servidor: String) {
        this.servidor = normalizarServidor(servidor)
        geracao++
    }
    fun aceita(senha: Int): Boolean = senha == geracao
}
```

android/app/src/main/java/br/com/viaagora/Modelos.kt

```
package br.com.viaagora

import java.time.Duration
import java.time.Instant

// Modelos de dados do app: travessias, situação de cada lado, relatos, categorias e histórico.
data class Terminal(val nome: String, val endereco: String, val latitude: Double, val longitude: Double)
data class SituacaoTravessia(
    val esperaA: Int, val esperaB: Int, val embarcacoes: Int, val operando: Boolean, val climaA: String?, val
climaB: String?,
    val periodo: String?, val aviso: String?, val fonte: String, val coletadoEm: String, val atrasadoNoServidor:
Boolean,
) {
    // Atrasado se o servidor disse ou se a leitura já tem mais de 10 min (por exemplo, cache aberto sem
internet).
    val atrasado: Boolean get() = atrasadoNoServidor ||
        runCatching { Duration.between(Instant.parse(coletadoEm), Instant.now()) > Duration.ofMinutes(10)
}.getOrDefault(true)
}
data class EsperaComunidade(val minutosEspera: Int, val relatos: Int)
data class Travessia(
    val id: String, val nome: String, val tipo: String, val pontoA: Terminal, val pontoB: Terminal,
    val situacao: SituacaoTravessia?, val comunidadeA: EsperaComunidade?, val comunidadeB: EsperaComunidade?, val
relatosAtivos: Int,
    val relatosParada: Int = 0, val rota: List<Pair<Double, Double>> = emptyList(),
) {
    fun terminal(lado: String) = if (lado == "A") pontoA else pontoB
    // Parada pelo painel oficial (sem embarcação ou aviso de interrupção).
    val parada: Boolean get() = situacao?.operando == false
}
data class InfoFonte(val modo: String?, val disponivel: Boolean, val ultimoSucessoEm: String?, val ultimoErro:
String?)
data class Panorama(val travessias: List<Travessia>, val relatos: List<Relato>, val fonte: InfoFonte)
```

```
data class Relato(
    val id: String, val autorId: String, val travessiaId: String, val lado: String, val categoria: String, val minutosEspera: Int?,
    val descricao: String, val latitude: Double, val longitude: Double, val situacao: String, val versao: Int,
    val criadoEm: String, val atualizadoEm: String,
)
data class MediaPorHora(val hora: Int, val esperaA: Int, val esperaB: Int)
data class Historico(val travessiaId: String, val porHora: List<MediaPorHora>, val melhorA: Int?, val melhorB: Int?)
data class Categoria(val key: String, val rotulo: String, val simbolo: String, val color: String)

val categorias = listOf(
    Categoria("fila", "Fila grande", "≡", "#C2410C"),
    Categoria("embarque_lento", "Embarque lento", "◷", "#B45309"),
    Categoria("interrompida", "Travessia parada", "■", "#B91C1C"),
    Categoria("clima", "Neblina ou mar agitado", "≈", "#1D4ED8"),
    Categoria("acidente", "Acidente no acesso", "!", "#7C3AED"),
)
fun infoCategoria(key: String): Categoria = categorias.firstOrNull { it.key == key } ?: categorias.first()

// Mesmo catálogo do servidor (terminais e rotas das linhas de balsa do OpenStreetMap); usado enquanto o
// servidor não responde.
val catalogoTravessias = listOf(
    Travessia("santos-guaruja", "Santos ⇄ Guarujá", "veiculos",
        Terminal("Santos", "Praça Gago Coutinho, 525 - Ponta da Praia", -23.98692, -46.29379),
        Terminal("Guarujá", "Av. Ademar de Barros, 3300", -23.98832, -46.28944), null, null, null, 0,
        rota = listOf(-23.98692 to -46.29379, -23.98832 to -46.28944)),
    Travessia("santos-vicente-de-carvalho", "Santos ⇄ Vicente de Carvalho", "pedestres",
        Terminal("Santos", "Praça da República - Centro", -23.93171, -46.32451),
        Terminal("Vicente de Carvalho", "Av. Thiago Ferreira", -23.93964, -46.30827), null, null, null, 0,
        rota = listOf(-23.93171 to -46.32451, -23.93075 to -46.32423, -23.93068 to -46.32406, -23.93152 to -46.31825, -23.93313 to -46.31383,
            -23.93512 to -46.31194, -23.94001 to -46.31023, -23.93963 to -46.30835, -23.93964 to -46.30827)),
    Travessia("bertioga-guaruja", "Bertioga ⇄ Guarujá", "veiculos",
        Terminal("Bertioga", "Av. Henrique Costabile, 50 - Jardim Veleiros", -23.85639, -46.14465),
        Terminal("Guarujá", "Estrada Guarujá-Bertioga, km 29", -23.86028, -46.14337), null, null, null, 0,
        rota = listOf(-23.85639 to -46.14465, -23.86028 to -46.14337)),
)

enum class NivelEspera(val rotulo: String, val color: String) {
    BAIXA("Tranquila", "#15803D"), MEDIA("Moderada", "#B45309"), ALTA("Longa", "#B91C1C"), FECHADA("Sem operação", "#64748B"), DESCONHECIDA("Sem dados", "#64748B")
}
fun nivelEspera(minutos: Int?, operando: Boolean = true): NivelEspera = when {
    !operando -> NivelEspera.FECHADA
    minutos == null -> NivelEspera.DESCONHECIDA
    minutos <= 20 -> NivelEspera.BAIXA
    minutos <= 45 -> NivelEspera.MEDIA
    else -> NivelEspera.ALTA
}
fun rotuloTipo(tipo: String) = if (tipo == "pedestres") "Pedestres e ciclistas" else "Veículos e pedestres"

fun localizacaoRecente(idadeMillis: Long): Boolean = idadeMillis in 0L..120_000L
fun validarRelato(travessiaId: String, lado: String, categoria: String, minutosEspera: Int?, descricao: String, latitude: Double, longitude: Double): String? = when {
    catalogoTravessias.none { it.id == travessiaId } -> "Escolha a travessia."
    lado !in listOf("A", "B") -> "Escolha de que lado você está."
    categorias.none { it.key == categoria } -> "Escolha o tipo de relato."
    minutosEspera != null && minutosEspera !in 0..600 -> "A espera deve ficar entre 0 e 600 minutos."
    descricao.trim().length > 400 -> "Use no máximo 400 caracteres."
    !latitude.isFinite() || latitude !in -90.0..90.0 || !longitude.isFinite() || longitude !in -180.0..180.0 -> "Escolha um ponto válido no mapa."
    else -> null
}
fun filtrarRelatos(dados: List<Relato>, travessiaId: String?, incluirEncerrados: Boolean): List<Relato> =
    dados.filter { (travessiaId == null || it.travessiaId == travessiaId) && (incluirEncerrados || it.situacao == "active") }
```

android/app/src/main/java/br/com/viaagora/LeituraJson.kt

```
package br.com.viaagora

import org.json.JSONArray
import org.json.JSONObject

// Conversão entre o JSON da API e os modelos do app. Os nomes entre aspas são os campos do contrato com o servidor.
// Dados de um relato que ainda vai ser enviado.
data class NovoRelato(
    val travessiaId: String, val lado: String, val categoria: String, val minutosEspera: Int?,
    val descricao: String, val latitude: Double, val longitude: Double,
) {
    fun json(): JSONObject = JSONObject().put("crossingId", travessiaId).put("side", lado).put("category", categoria)
```

```
13          .put("waitMinutes", minutosEspera ?: JSONObject.NULL).put("description", descricao.trim())
14          .put("latitude", latitude).put("longitude", longitude)
15  }
16  private fun JSONObject.text(nome: String): String? = if (isNull(nome)) null else optString(nome)
17  private fun JSONObject.numero(nome: String): Int? = if (isNull(nome) || !has(nome)) null else getInt(nome)
18  private fun <T> JSONArray?.itens(ler: (JSONObject) -> T): List<T> =
19      if (this == null) emptyList() else (0 until length()).mapNotNull { runCatching { ler(getJSONObject(it))
}.getOrNull() }
20
21  fun Relato.json(): JSONObject = JSONObject().put("id", id).put("authorId", autorId).put("crossingId",
travessiaId).put("side", lado)
22      .put("category", categoria).put("waitMinutes", minutosEspera ?: JSONObject.NULL).put("description",
descricao)
23      .put("latitude", latitude).put("longitude", longitude).put("status", situacao).put("version", versao)
24      .put("createdAt", criadoEm).put("updatedAt", atualizadoEm)
25  fun lerRelato(o: JSONObject) = Relato(o.getString("id"), o.getString("authorId"), o.getString("crossingId"),
o.getString("side"),
26      o.getString("category"), o.numero("waitMinutes"), o.optString("description"), o.getDouble("latitude"),
o.getDouble("longitude"),
27      o.getString("status"), o.getInt("version"), o.getString("createdAt"), o.getString("updatedAt"))
28  fun codificarRelatos(dados: List<Relato>): String = JSONArray().apply { dados.forEach { put(it.json()) }
}.toString()
29  fun lerRelatos(text: String): List<Relato> = runCatching { JSONArray(text).itens(::lerRelato)
}.getOrDefault(emptyList())
30
31  private fun lerTerminal(o: JSONObject) = Terminal(o.getString("name"), o.optString("address"),
o.getDouble("latitude"), o.getDouble("longitude"))
32  private fun lerComunidade(o: JSONObject?) = o?.let { EsperaComunidade(it.getInt("waitMinutes"),
it.getInt("reports")) }
33  private fun lerSituacao(o: JSONObject?) = o?.let {
34      SituacaoTravessia(it.getInt("waitA"), it.getInt("waitB"), it.getInt("boats"), it.getBoolean("operating"),
it.text("weatherA"), it.text("weatherB"),
35          it.text("period"), it.text("notice"), it.getString("source"), it.getString("collectedAt"),
it.optBoolean("stale"))
36  }
37  fun lerTravessia(o: JSONObject): Travessia {
38      val comunidade = o.optJSONObject("community")
39      val id = o.getString("id")
40      val pontos = o.optJSONArray("route")
41      val rota = if (pontos == null) catalogoTravessias.firstOrNull { it.id == id }?.rota.orEmpty()
42          else (0 until pontos.length()).map { pontos.getJSONArray(it).let { p -> p.getDouble(0) to p.getDouble(1)
} }
43      return Travessia(id, o.getString("name"), o.getString("kind"), lerTerminal(o.getJSONObject("pointA")),
44          lerTerminal(o.getJSONObject("pointB")), lerSituacao(o.optJSONObject("status")),
lerComunidade(comunidade?.optJSONObject("A")),
45          lerComunidade(comunidade?.optJSONObject("B")), o.optInt("activeReports"), o.optInt("stopReports"), rota)
46  }
47  fun lerTravessias(text: String): Pair<List<Travessia>, InfoFonte> {
48      val o = JSONObject(text)
49      val s = o.optJSONObject("source") ?: JSONObject()
50      return o.optJSONArray("crossings").itens(::lerTravessia) to
51          InfoFonte(s.text("mode"), s.optBoolean("available"), s.text("lastSuccessAt"), s.text("lastError"))
52  }
53  fun lerHistorico(text: String): Historico {
54      val o = JSONObject(text)
55      val melhor = o.optJSONObject("bestHour") ?: JSONObject()
56      return Historico(o.getString("crossingId"),
57          o.optJSONArray("hourly").itens { MediaPorHora(it.getInt("hour"), it.getInt("waitA"), it.getInt("waitB"))
}, melhor.numero("A"), melhor.numero("B"))
58  }
```

android/app/src/main/java/br/com/viaagora/EstadoSistema.kt

```
 1  package br.com.viaagora
 2
 3  import org.json.JSONObject
 4  import java.time.Duration
 5  import java.time.Instant
 6
 7  // Dados da rota /v1/system, mostrados na aba Bastidores.
 8  data class EstadoProcesso(val processo: String, val papel: String, val vivo: Boolean, val idadeMs: Long, val
celulares: Int?, val broker: Boolean?, val coletorOk: Boolean?)
 9  data class Entrega(val replica: String, val celulares: Int, val entregueEm: String)
10  data class RastroEvento(
11      val eventoId: String, val tipo: String, val origem: String?, val travessiaId: String?,
12      val criadoEm: String, val publicadoEm: String?, val entregas: List<Entrega>,
13  )
14  data class EstadoSistema(
15      val atendidoPor: String, val processos: List<EstadoProcesso>, val brokerNoAr: Boolean, val pendentes: Int,
val ultimaHora: Int,
16      val reenvios: Int, val coletas24h: Int, val eventos: List<RastroEvento>,
17  )
18
19  fun lerSistema(text: String): EstadoSistema {
20      val o = JSONObject(text)
```

```
 21      val processos = o.optJSONArray("nodes")?.let { a -> (0 until a.length()).map { a.getJSONObject(it) }
}.orEmpty().map { n ->
 22          val info = n.optJSONObject("info") ?: JSONObject()
 23          EstadoProcesso(n.getString("node"), n.getString("role"), n.optBoolean("alive"), n.optLong("ageMs"),
 24              if (info.has("clients")) info.optInt("clients") else null, if (info.has("broker"))
info.optBoolean("broker") else null,
 25              if (info.has("lastOk") && !info.isNull("lastOk")) info.optBoolean("lastOk") else null)
 26      }
 27      val eventos = o.optJSONArray("events")?.let { a -> (0 until a.length()).map { a.getJSONObject(it) }
}.orEmpty().map { e ->
 28          val d = e.optJSONArray("deliveries")
 29          RastroEvento(e.getString("eventId"), e.optString("type"), e.optString("origin").takeIf { it.isNotBlank()
&& it != "null" },
 30              e.optString("crossingId").takeIf { it.isNotBlank() && it != "null" }, e.getString("createdAt"),
 31              e.optString("publishedAt").takeIf { it.isNotBlank() && it != "null" },
 32              (0 until (d?.length() ?: 0)).map { i -> d!!.getJSONObject(i).let { Entrega(it.getString("replica"),
it.optInt("clients"), it.getString("deliveredAt")) } })
 33      }
 34      val outbox = o.optJSONObject("outbox") ?: JSONObject()
 35      return EstadoSistema(o.optString("servedBy"), processos, o.optJSONObject("services")?.optString("broker") ==
"up",
 36          outbox.optInt("pending"), outbox.optInt("lastHour"),
o.optJSONObject("counters")?.optInt("idempotentReplays") ?: 0,
 37          o.optJSONObject("collections")?.optInt("last24h") ?: 0, eventos)
 38  }
 39
 40  // Nó do desenho que representa cada réplica e o grupo de celulares ligado a ela.
 41  fun chaveReplica(nome: String?) = if (nome?.endsWith("2") == true) "api2" else "api1"
 42  fun chaveCelulares(chaveReplica: String) = if (chaveReplica == "api2") "cel2" else "cel1"
 43
 44  data class PassoRastro(val saltos: List<Pair<String, String>>, val rotulo: String, val legenda: String, val
conceitos: List<String>, val ms: Long?)
 45
 46  private fun milissegundos(from: String?, to: String?): Long? = runCatching {
Duration.between(Instant.parse(from), Instant.parse(to)).toMillis().coerceAtLeast(0) }.getOrNull()
 47
 48  // Transforma o rastro de um evento real nos passos que a animação percorre, com o tempo medido de cada um.
 49  fun passosDoRastro(evento: RastroEvento, now: Instant = Instant.now()): List<PassoRastro> {
 50      val doColetor = evento.origem == "collector" || (evento.origem == null && !evento.tipo.startsWith("report"))
 51      val passos = mutableListOf<PassoRastro>()
 52      if (doColetor) {
 53          passos += PassoRastro(listOf("semil" to "col"), "Painel oficial → coletor", "O coletor leu o painel
oficial: um único acesso por minuto, para todos os usuários.", listOf("HTTP"), null)
 54          passos += PassoRastro(listOf("col" to "db"), "Coletor → banco + outbox", "Algo mudou, então a leitura e
o evento foram gravados juntos, numa transação.", listOf("Outbox"), null)
 55      } else {
 56          val api = chaveReplica(evento.origem); val nome = evento.origem ?: "API"
 57          passos += PassoRastro(listOf(chaveCelulares(api) to api), "Celular → Nginx → $nome", "Um celular enviou
um relato por HTTPS. O Nginx, que reparte os acessos, o entregou à $nome.", listOf("HTTP", "Balanceamento"), null)
 58          passos += PassoRastro(listOf(api to "db"), "$nome → banco + outbox", "A $nome gravou o relato e o evento
juntos, numa transação: ou salva os dois, ou nenhum.", listOf("Outbox"), null)
 59      }
 60      passos += PassoRastro(listOf("db" to "wk"), "Outbox → worker", "O worker encontrou o evento pendente na
tabela outbox.", listOf("Outbox"), null)
 61      if (evento.publicadoEm == null) {
 62          passos += PassoRastro(emptyList(), "Aguardando o RabbitMQ", "O evento ainda não foi publicado: espera na
outbox. Nada se perde, mesmo com o broker fora do ar.", listOf("Outbox", "Tolerância a falhas"), null)
 63          return passos
 64      }
 65      passos += PassoRastro(listOf("wk" to "mq"), "Worker → RabbitMQ", "O worker publicou no RabbitMQ e só marcou
como enviado depois da confirmação.", listOf("Pub/Sub"), milissegundos(evento.criadoEm, evento.publicadoEm))
 66      val entregues = evento.entregas.sortedBy { it.replica }
 67      if (entregues.isEmpty()) {
 68          // Publicado enquanto as réplicas reconectavam ao broker: ninguém recebeu esta cópia, mas a
ressincronização cobre.
 69          if ((milissegundos(evento.publicadoEm, now.toString()) ?: 0) > 5_000) passos += PassoRastro(emptyList(),
"Entregue por ressincronização",
 70              "As réplicas ainda reconectavam ao broker quando o evento saiu. Ao voltar, elas pedem aos celulares
que busquem tudo de novo: ninguém fica desatualizado.",
 71              listOf("Tolerância a falhas"), null)
 72          return passos
 73      }
 74      val msFanout = entregues.mapNotNull { milissegundos(evento.publicadoEm, it.entregueEm) }.maxOrNull()
 75      passos += PassoRastro(entregues.map { "mq" to chaveReplica(it.replica) }, "RabbitMQ → " +
entregues.joinToString(" e ") { it.replica },
 76          "A exchange fanout entregou uma cópia para cada réplica: todas ficam sabendo do mesmo evento.",
listOf("Pub/Sub", "Replicação"), msFanout)
 77      val celulares = entregues.sumOf { it.celulares }
 78      passos += PassoRastro(entregues.map { chaveReplica(it.replica) to chaveCelulares(chaveReplica(it.replica))
}, "Réplicas → celulares",
 79          "Cada réplica avisou na hora, pelo WebSocket, os celulares ligados a ela ($celulares conectado${if
(celulares == 1) "" else "s"}).", listOf("WebSocket"), null)
 80      return passos
 81  }
```

```
 82  fun tempoTotal(evento: RastroEvento): Long? = milissegundos(evento.criadoEm, evento.entregas.maxOfOrNull {
it.entregueEm } ?: evento.publicadoEm)
 83
 84  fun tituloEvento(tipo: String) = when (tipo) {
 85      "report.created" -> "Relato publicado"
 86      "report.resolved" -> "Relato encerrado"
 87      "crossing.updated" -> "Nova leitura do painel"
 88      "source.status" -> "Fonte oficial mudou de estado"
 89      else -> "Evento"
 90  }
 91
 92  // Exemplo com tempos típicos, mostrado antes do primeiro evento real.
 93  fun rastroExemplo(now: Instant = Instant.now()) = RastroEvento("exemplo", "report.created", "api-2", "santos-
guaruja",
 94      now.minusMillis(400).toString(), now.minusMillis(10).toString(),
 95      listOf(Entrega("api-1", 3, now.minusMillis(6).toString()), Entrega("api-2", 2,
now.minusMillis(5).toString())))
 96
 97  data class Explicacao(val titulo: String, val oQue: String, val conceito: String, val seCair: String?)
 98  val explicacoesProcessos = mapOf(
 99      "semil" to Explicacao("Site SEMIL", "O painel público das Travessias Litorâneas. Não é nosso: o sistema
precisa conviver com ele lento, fora do ar ou mudando.", "HTTP", "O app continua mostrando o último dado, marcado
como atrasado."),
100      "col" to Explicacao("Coletor", "Um único processo lê o painel a cada minuto para todos os usuários. Assim o
site oficial não recebe um acesso por celular.", "Heartbeat", "Tenta de novo esperando cada vez mais, até 10
minutos."),
101      "db" to Explicacao("Banco (PostgreSQL)", "A fonte da verdade. Tudo é gravado aqui antes de qualquer aviso
sair.", "Outbox", "As APIs recusam novos relatos até ele voltar: nada é aceito sem ser salvo."),
102      "wk" to Explicacao("Worker", "Lê a tabela outbox e publica os eventos pendentes no RabbitMQ, esperando a
confirmação.", "Outbox", "Os eventos esperam na outbox até ele voltar."),
103      "mq" to Explicacao("RabbitMQ", "O broker de mensagens. Recebe cada evento uma vez e entrega uma cópia para
cada réplica da API (exchange fanout).", "Pub/Sub", "Os relatos continuam sendo salvos; os avisos chegam quando ele
voltar."),
104      "nginx" to Explicacao("Nginx (balanceador)", "A porta de entrada do servidor. Recebe todo acesso HTTPS e
WebSocket dos celulares e reparte entre as duas réplicas da API.", "Balanceamento", "O app fica sem conexão e mostra
o último dado salvo até ele voltar."),
105      "api1" to Explicacao("API réplica 1", "Uma de duas cópias idênticas da API. O Nginx divide os celulares
entre elas.", "Replicação", "Os celulares dela reconectam na outra réplica, que continua atendendo."),
106      "api2" to Explicacao("API réplica 2", "A outra cópia da API, com o mesmo código e o mesmo banco.",
"Replicação", "Os celulares dela reconectam na outra réplica, que continua atendendo."),
107      "cel1" to Explicacao("Celulares da réplica 1", "Aparelhos com o canal ao vivo (WebSocket) aberto na réplica
1.", "WebSocket", null),
108      "cel2" to Explicacao("Celulares da réplica 2", "Aparelhos com o canal ao vivo (WebSocket) aberto na réplica
2.", "WebSocket", null),
109  )
110  val explicacoesConceitos = linkedMapOf(
111      "HTTP" to "Pedido e resposta: o celular pede ou envia dados e a resposta volta na mesma conexão.",
112      "Outbox" to "O evento é gravado na mesma transação do dado. Se o broker cair, nada se perde: o worker
publica depois.",
113      "Pub/Sub" to "Quem publica não sabe quem vai receber. O RabbitMQ entrega uma cópia para cada interessado.",
114      "Balanceamento" to "Um único endereço na frente de vários servidores: o Nginx reparte os pedidos e, se uma
réplica cai, manda tudo para a outra.",
115      "Replicação" to "Duas cópias iguais da API dividem os celulares. Se uma cair, a outra continua atendendo.",
116      "WebSocket" to "Uma conexão que fica aberta: o servidor avisa o celular na hora, sem o celular ficar
perguntando.",
117      "Heartbeat" to "Cada processo avisa \"estou vivo\" a cada 5 segundos. Quem para de avisar aparece como fora
do ar.",
118      "Idempotência" to "Enviar o mesmo relato de novo (por exemplo, após falha de rede) não cria um relato
duplicado.",
119      "Tolerância a falhas" to "O sistema continua funcionando quando uma parte falha e se recupera sozinho quando
ela volta.",
120  )
```

android/app/src/main/java/br/com/viaagora/PaginaBastidores.kt

```
 1  package br.com.viaagora
 2
 3  import android.content.Context
 4  import android.graphics.Color
 5  import android.os.Handler
 6  import android.os.Looper
 7  import android.text.SpannableStringBuilder
 8  import android.text.Spanned
 9  import android.text.style.ForegroundColorSpan
10  import android.view.Gravity
11  import android.view.View
12  import android.widget.*
13  import java.time.Duration
14  import java.time.Instant
15
16  // Aba Bastidores: mostra o sistema distribuído funcionando, com os dados reais de /v1/system.
17  class PaginaBastidores(private val context: Context, private val ui: Ui) {
18      // Quem fornece o estado do servidor (a rota /v1/system).
19      var fonte: (((EstadoSistema) -> Unit, (String) -> Unit) -> Unit)? = null
20      var minhaReplica: String? = null
```

```
21      // Endereço do servidor conectado: a VPS oficial ou o computador de quem roda o sistema localmente.
22      var servidor: String = SERVIDOR_OFICIAL
23          set(valor) { field = valor; diagrama.rotuloServidor = if (valor == SERVIDOR_OFICIAL) "VPS · 
SEDDA.COM.BR" else "SERVIDOR · ${valor.substringAfter("://").uppercase()}" }
24      private val agendador = Handler(Looper.getMainLooper())
25      private val conteudo = ui.coluna().apply { setPadding(context.dp(16), context.dp(4), context.dp(16), 
context.dp(96)) }
26      val view = ScrollView(context).apply { addView(conteudo) }
27      private val introducao = ui.text("", 14f, Paleta.apagado).apply { setLineSpacing(0f, 1.15f) }
28      private val diagrama = DiagramaRede(context, ::explicarProcesso)
29      private val etiquetaPasso = ui.text("", 11f, Paleta.destaque, true).apply { letterSpacing = 0.08f }
30      private val legenda = ui.text("", 15f, Paleta.tinta).apply { setLineSpacing(0f, 1.2f); minLines = 3 }
31      private val etiquetas = linkedMapOf<String, TextView>()
32      private val acoes = ui.linha()
33      private val tituloLinhaTempo = ui.text("", 13f, Paleta.apagado)
34      private val linhaTempo = ui.coluna()
35      private val metricas = ui.linha()
36      private var estado: EstadoSistema? = null
37      private var ultimoEvento: RastroEvento? = null
38      private var ultimoAnimado: String? = null
39      private var animando = false
40      private var aula: List<PassoRastro>? = null
41      private var passoDaAula = 0
42      private var visivel = false
43      private val consulta = object : Runnable { override fun run() { recarregar(); agendador.postDelayed(this, 
5_000) } }
44
45      init {
46          conteudo.addView(ui.text("SISTEMA DISTRIBUÍDO", 11f, Paleta.destaque, true).apply { letterSpacing = 
0.1f; setPadding(0, context.dp(8), 0, context.dp(6)) })
47          conteudo.addView(ui.text("Por dentro do ViaAgora", 24f, Paleta.tinta, true))
48          conteudo.addView(ui.espaco(6))
49          conteudo.addView(introducao)
50          conteudo.addView(ui.espaco(16))
51          val quadroDiagrama = ui.coluna().apply { setPadding(context.dp(10), context.dp(12), context.dp(10), 
context.dp(14)); background = ui.arredondado(Color.WHITE, 22) }
52          quadroDiagrama.addView(diagrama)
53          quadroDiagrama.addView(ui.text(textoLegenda(), 12f, Paleta.apagado).apply { gravity = Gravity.CENTER; 
setPadding(0, context.dp(10), 0, 0) })
54          conteudo.addView(quadroDiagrama)
55          conteudo.addView(ui.espaco(12))
56          val explicacao = ui.coluna().apply { setPadding(context.dp(18), context.dp(14), context.dp(18), 
context.dp(14)); background = ui.arredondado(Paleta.suave, 18) }
57          explicacao.addView(etiquetaPasso); explicacao.addView(ui.espaco(6)); explicacao.addView(legenda)
58          conteudo.addView(explicacao)
59          conteudo.addView(ui.espaco(12))
60          val linhaEtiquetas = ui.linha()
61          explicacoesConceitos.keys.forEach { conceito ->
62              val etiqueta = ui.etiqueta(conceito, false) { explicarConceito(conceito) }
63              etiquetas[conceito] = etiqueta
64              linhaEtiquetas.addView(etiqueta, LinearLayout.LayoutParams(LinearLayout.LayoutParams.WRAP_CONTENT, 
context.dp(36)).apply { marginEnd = context.dp(6) })
65          }
66          conteudo.addView(HorizontalScrollView(context).apply { isHorizontalScrollBarEnabled = false; 
addView(linhaEtiquetas) })
67          conteudo.addView(ui.espaco(14))
68          conteudo.addView(acoes)
69          conteudo.addView(secao("O caminho de um evento"))
70          conteudo.addView(tituloLinhaTempo)
71          conteudo.addView(ui.espaco(8))
72          conteudo.addView(linhaTempo)
73          conteudo.addView(secao("Agora no servidor"))
74          conteudo.addView(metricas)
75          legendaInicial()
76          mostrarAcoes()
77      }
78
79      fun show() { if (visivel) return; visivel = true; agendador.post(consulta) }
80      fun hide() { visivel = false; agendador.removeCallbacks(consulta) }
81      fun reiniciar() { estado = null; ultimoEvento = null; ultimoAnimado = null; minhaReplica = null; aula = 
null; diagrama.stop(); animando = false; atualizarTela(); if (visivel) agendador.postDelayed({ recarregar() }, 300) }
82
83      // Evento ao vivo do WebSocket: a réplica se apresenta no "ready"; nos demais, buscamos o rastro completo.
84      fun aoEventoAoVivo(tipo: String, replica: String?, temEventoId: Boolean) {
85          if (tipo == "ready" && replica != null) { minhaReplica = replica; atualizarTela() }
86          if (temEventoId && visivel) agendador.postDelayed({ recarregar() }, 1200)
87      }
88
89      private fun recarregar() {
90          val carregar = fonte
91          if (carregar == null) return
92          carregar({ s -> estado = s; s.eventos.firstOrNull()?.let { ultimoEvento = it }; atualizarTela(); 
animarAutomatico() }, { if (visivel) { estado = null; atualizarTela() } })
93      }
94      private fun animarAutomatico() {
```

```
 95          val evento = ultimoEvento ?: return
 96          // A chave muda quando o mesmo evento avança (ex.: estava preso na outbox e o broker voltou), e então
reanimamos.
 97          val key = "${evento.eventoId}:${evento.publicadoEm != null}:${evento.entregas.size}"
 98          if (!visivel || aula != null || animando || key == ultimoAnimado) return
 99          val retomado = ultimoAnimado?.startsWith("${evento.eventoId}:false") == true
100          ultimoAnimado = key
101          val passos = passosDoRastro(evento)
102          if (retomado) {
103              val start = passos.indexOfFirst { it.saltos == listOf("wk" to "mq") }.coerceAtLeast(0)
104              etiquetaPasso.text = "RECUPERAÇÃO"; legenda.text = "O broker voltou: o worker publica o evento que
estava guardado na outbox."
105              agendador.postDelayed({ animarTudo(passos, corDoEvento(evento), start) }, 1500)
106          } else animarTudo(passos, corDoEvento(evento))
107      }
108      private fun corDoEvento(evento: RastroEvento) = Color.parseColor(if (evento.tipo.startsWith("report"))
"#D85A30" else "#378ADD")
109      private fun animarTudo(passos: List<PassoRastro>, color: Int, indice: Int = 0) {
110          if (indice >= passos.size || !visivel) { animando = false; return }
111          animando = true
112          val passo = passos[indice]
113          mostrarPasso(passo, "Passo ${indice + 1} de ${passos.size}")
114          diagrama.animar(passo.saltos, color) { agendador.postDelayed({ animarTudo(passos, color, indice + 1) },
450) }
115      }
116      private fun mostrarPasso(passo: PassoRastro, posicao: String) {
117          etiquetaPasso.text = posicao.uppercase(); legenda.text = passo.legenda
118          etiquetas.forEach { (conceito, etiqueta) ->
119              val ativo = conceito in passo.conceitos
120              etiqueta.background = ui.arredondado(if (ativo) Paleta.destaque else Paleta.suave, 20);
etiqueta.setTextColor(if (ativo) Color.WHITE else Paleta.tinta)
121          }
122      }
123
124      private fun atualizarTela() {
125          val s = estado
126          introducao.text = when {
127              s == null -> "Conectando ao servidor…"
128              else -> "Estes processos trabalham juntos no servidor. Seu celular está ligado à ${minhaReplica?.let
{ "réplica $it" } ?: "uma das réplicas"}, destacada no desenho."
129          }
130          diagrama.saude = saude(s)
131          diagrama.voce = minhaReplica?.let { chaveReplica(it) }
132          diagrama.rotulosCelulares = listOf("api1", "api2").associate { key ->
133              val celulares = s?.processos?.firstOrNull { it.papel == "api" && chaveReplica(it.processo) == key
}?.celulares
134              val meu = minhaReplica != null && chaveReplica(minhaReplica) == key
135              chaveCelulares(key) to when {
136                  celulares == null -> if (meu) "Você" else "Celulares"
137                  meu -> if (celulares > 1) "Você +${celulares - 1}" else "Você"
138                  else -> "$celulares celular${if (celulares == 1) "" else "es"}"
139              }
140          }
141          mostrarLinhaTempo()
142          if (aula == null) mostrarAcoes()
143          metricas.removeAllViews()
144          listOf(Triple("bolt", "eventos na última hora", s?.ultimaHora), Triple("queue", "esperando na outbox",
s?.pendentes), Triple("repeat", "reenvios ignorados", s?.reenvios))
145              .forEachIndexed { i, (nome, rotulo, valor) ->
146                  val caixa = ui.coluna().apply { setPadding(context.dp(12), context.dp(12), context.dp(12),
context.dp(12)); background = ui.arredondado(Color.WHITE, 16); isClickable = true }
147                  caixa.addView(icone(nome, 22, Paleta.destaque))
148                  caixa.addView(ui.text(valor?.toString() ?: "—", 22f, Paleta.tinta, true).apply { setPadding(0,
context.dp(10), 0, 0) })
149                  caixa.addView(ui.text(rotulo, 11f, Paleta.apagado).apply { setPadding(0, context.dp(4), 0, 0) })
150                  caixa.setOnClickListener { explicarConceito(listOf("Pub/Sub", "Outbox", "Idempotência")[i]) }
151                  metricas.addView(caixa, LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.MATCH_PARENT,
1f).apply { if (i > 0) marginStart = context.dp(8) })
152              }
153      }
154      private fun saude(s: EstadoSistema?): Map<String, DiagramaRede.Saude> {
155          if (s == null) return emptyMap()
156          fun saudeDe(vivo: Boolean?) = when (vivo) { true -> DiagramaRede.Saude.NO_AR; false ->
DiagramaRede.Saude.FORA_DO_AR; null -> DiagramaRede.Saude.DESCONHECIDA }
157          val apisVivas = s.processos.filter { it.papel == "api" }
158          val coletor = s.processos.firstOrNull { it.papel == "collector" }
159          return mapOf(
160              "api1" to saudeDe(apisVivas.firstOrNull { chaveReplica(it.processo) == "api1" }?.vivo), "api2" to
saudeDe(apisVivas.firstOrNull { chaveReplica(it.processo) == "api2" }?.vivo),
161              "wk" to saudeDe(s.processos.firstOrNull { it.papel == "worker" }?.vivo), "col" to
saudeDe(coletor?.vivo),
162              "db" to DiagramaRede.Saude.NO_AR, "nginx" to DiagramaRede.Saude.NO_AR, "mq" to
saudeDe(s.brokerNoAr), "semil" to saudeDe(coletor?.coletorOk),
163              "cel1" to saudeDe(apisVivas.firstOrNull { chaveReplica(it.processo) == "api1" }?.celulares?.let { it
> 0 }?.takeIf { it }),
```

```
            "cel2" to saudeDe(apisVivas.firstOrNull { chaveReplica(it.processo) == "api2" }?.celulares?.let { it
> 0 }?.takeIf { it }),
        )
    }
    private fun mostrarLinhaTempo() {
        linhaTempo.removeAllViews()
        val evento = ultimoEvento
        if (evento == null) { tituloLinhaTempo.text = "Nenhum evento ainda. Publique um relato e volte aqui.";
return }
        tituloLinhaTempo.text = if (evento.eventoId == "exemplo") "Exemplo · relato publicado" else
"${tituloEvento(evento.tipo)} · ${haQuanto(evento.criadoEm)}"
        val caixa = ui.coluna().apply { setPadding(context.dp(16), context.dp(10), context.dp(16),
context.dp(10)); background = ui.arredondado(Color.WHITE, 18) }
        passosDoRastro(evento).forEach { passo ->
            val linha = ui.linha().apply { setPadding(0, context.dp(7), 0, context.dp(7)) }
            val nome = passo.saltos.firstOrNull()?.second?.let { DiagramaRede.iconeDe(it) } ?: if
(passo.rotulo.startsWith("Entregue")) "repeat" else "clock"
            linha.addView(icone(nome, 18, Paleta.destaque).apply { background = ui.arredondado(Paleta.suave,
10); setPadding(context.dp(7), context.dp(7), context.dp(7), context.dp(7)) },
                LinearLayout.LayoutParams(context.dp(32), context.dp(32)).apply { marginEnd = context.dp(12) })
            linha.addView(ui.text(passo.rotulo, 14f, Paleta.tinta), LinearLayout.LayoutParams(0,
LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
            linha.addView(ui.text(passo.ms?.let { "+$it ms" } ?: "", 13f, Paleta.apagado))
            caixa.addView(linha)
        }
        tempoTotal(evento)?.takeIf { evento.entregas.isNotEmpty() }?.let { total ->
            caixa.addView(View(context).apply { setBackgroundColor(Paleta.linha) },
LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, 1).apply { topMargin = context.dp(6) })
            val linha = ui.linha().apply { setPadding(0, context.dp(12), 0, context.dp(4)) }
            linha.addView(ui.text("Do banco até os celulares", 14f, Paleta.tinta, true),
LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
            linha.addView(ui.text("$total ms", 15f, Paleta.destaque, true))
            caixa.addView(linha)
        }
        linhaTempo.addView(caixa)
    }
    private fun mostrarAcoes() {
        acoes.removeAllViews()
        val passos = aula
        val botoes = if (passos == null) listOf(
            "▶  Assistir" to { repetir() },
            "Passo a passo" to { iniciarAula() },
        ) else listOf(
            "Voltar" to { irParaPasso(passoDaAula - 1) },
            (if (passoDaAula >= passos.size - 1) "Concluir" else "Próximo") to { if (passoDaAula >= passos.size
- 1) encerrarAula() else irParaPasso(passoDaAula + 1) },
        )
        botoes.forEachIndexed { i, (rotulo, action) ->
            acoes.addView(ui.botao(rotulo, if (passos == null) i == 0 else i == 1, action),
LinearLayout.LayoutParams(0, context.dp(46), 1f).apply { if (i > 0) marginStart = context.dp(8) })
        }
    }
    private fun repetir() {
        if (animando) return
        val evento = ultimoEvento ?: rastroExemplo().also { ultimoEvento = it; atualizarTela() }
        animarTudo(passosDoRastro(evento), corDoEvento(evento))
    }
    // Modo aula: o pacote anda um salto por vez, no ritmo de quem está explicando.
    private fun iniciarAula() {
        val evento = ultimoEvento ?: rastroExemplo().also { ultimoEvento = it; atualizarTela() }
        diagrama.stop(); animando = false
        aula = passosDoRastro(evento); irParaPasso(0)
    }
    private fun irParaPasso(indice: Int) {
        val passos = aula ?: return
        passoDaAula = indice.coerceIn(0, passos.size - 1)
        val passo = passos[passoDaAula]
        mostrarPasso(passo, "Passo ${passoDaAula + 1} de ${passos.size}")
        ultimoEvento?.let { diagrama.animar(passo.saltos, corDoEvento(it)) {} }
        mostrarAcoes()
    }
    private fun encerrarAula() { aula = null; legendaInicial(); mostrarAcoes() }
    private fun legendaInicial() {
        etiquetaPasso.text = "COMO LER O DESENHO"
        legenda.text = "Cada quadro é um processo. As bolinhas que andam são mensagens. Toque num quadro para
saber o que ele faz."
    }
    private fun secao(titulo: String) = ui.text(titulo, 17f, Paleta.tinta, true).apply { setPadding(0,
context.dp(28), 0, context.dp(6)) }
    private fun icone(nome: String, size: Int, color: Int) = ImageView(context).apply {
        setImageDrawable(DesenhoIcone(nome, color)); layoutParams = LinearLayout.LayoutParams(context.dp(size),
context.dp(size))
    }
    private fun textoLegenda(): CharSequence = SpannableStringBuilder().apply {
```

```
234          listOf(Color.parseColor("#16A34A") to "no ar", Paleta.perigo to "fora do ar",
Color.parseColor("#D85A30") to "relato", Color.parseColor("#378ADD") to "leitura do painel")
235              .forEachIndexed { i, (color, rotulo) ->
236                  if (i > 0) append("    ")
237                  val start = length; append("●"); setSpan(ForegroundColorSpan(color), start, length,
Spanned.SPAN_EXCLUSIVE_EXCLUSIVE); append(" $rotulo")
238              }
239      }
240
241      private fun explicarProcesso(key: String) {
242          val info = explicacoesProcessos[key] ?: return
243          val corpo = ui.coluna()
244          situacaoAoVivo(key)?.let { (text, color) -> corpo.addView(ui.text(text, 14f, color, true).apply {
setPadding(0, context.dp(4), 0, context.dp(10)) }) }
245          corpo.addView(ui.text(info.oQue, 15f, Paleta.tinta).apply { setLineSpacing(0f, 1.2f) })
246          info.seCair?.let { corpo.addView(ui.text("Se cair: $it", 14f, Paleta.apagado).apply { setPadding(0,
context.dp(12), 0, 0); setLineSpacing(0f, 1.2f) }) }
247          corpo.addView(ui.espaco(14))
248          corpo.addView(ui.text("Conceito: ${info.conceito}", 13f, Paleta.destaque, true))
249          corpo.addView(ui.text(explicacoesConceitos[info.conceito].orEmpty(), 13f, Paleta.apagado).apply {
setPadding(0, context.dp(4), 0, context.dp(8)) })
250          ui.painel(info.titulo, corpo).show()
251      }
252      private fun situacaoAoVivo(key: String): Pair<String, Int>? {
253          val s = estado ?: return null
254          val processo = when (key) {
255              "api1", "api2" -> s.processos.firstOrNull { it.papel == "api" && chaveReplica(it.processo) == key }
256              "wk" -> s.processos.firstOrNull { it.papel == "worker" }
257              "col" -> s.processos.firstOrNull { it.papel == "collector" }
258              else -> null
259          }
260          return when {
261              key == "mq" -> if (s.brokerNoAr) "● Conectado às réplicas" to Paleta.destaque else "● Fora do ar ·
${s.pendentes} evento(s) esperando na outbox" to Paleta.perigo
262              key == "db" -> "● Respondendo" to Paleta.destaque
263              key == "semil" -> when (s.processos.firstOrNull { it.papel == "collector" }?.coletorOk) { false ->
"● Última leitura falhou" to Paleta.alerta; true -> "● Última leitura OK" to Paleta.destaque; null -> null }
264              processo == null -> null
265              processo.vivo -> "● Vivo · último sinal há ${processo.idadeMs / 1000} s" + (processo.celulares?.let
{ " · $it celular(es)" } ?: "") to Paleta.destaque
266              else -> "● Sem sinal há ${processo.idadeMs / 1000} s" to Paleta.perigo
267          }
268      }
269      private fun explicarConceito(conceito: String) {
270          val corpo = ui.coluna()
271          corpo.addView(ui.text(explicacoesConceitos[conceito].orEmpty(), 15f, Paleta.tinta).apply {
setLineSpacing(0f, 1.2f); setPadding(0, context.dp(4), 0, context.dp(8)) })
272          if (conceito == "Idempotência") estado?.let { corpo.addView(ui.text("Reenvios ignorados até agora:
${it.reenvios}", 13f, Paleta.apagado)) }
273          ui.painel(conceito, corpo).show()
274      }
275      private fun haQuanto(valor: String) = runCatching {
276          val seconds = Duration.between(Instant.parse(valor), Instant.now()).seconds.coerceAtLeast(0)
277          if (seconds < 60) "há $seconds s" else if (seconds < 3600) "há ${seconds / 60} min" else "há ${seconds /
3600} h"
278      }.getOrDefault("")
279  }
```

android/app/src/main/java/br/com/viaagora/DiagramaRede.kt

```
 1  package br.com.viaagora
 2
 3  import android.annotation.SuppressLint
 4  import android.content.Context
 5  import android.graphics.*
 6  import android.os.SystemClock
 7  import android.view.MotionEvent
 8  import android.view.View
 9
10  // Diagrama do sistema: a internet em cima e, dentro da VPS, os processos. Desenhado num espaço de 340 x 474
11  // e escalado para a largura da tela.
12  @SuppressLint("ViewConstructor")
13  class DiagramaRede(context: Context, private val aoTocarProcesso: (String) -> Unit) : View(context) {
14      enum class Saude { DESCONHECIDA, NO_AR, FORA_DO_AR }
15      private data class Quadro(val x: Float, val y: Float, val rotulo: String)
16      private val processos = linkedMapOf(
17          "semil" to Quadro(46f, 62f, "Site SEMIL"),
18          "cel1" to Quadro(145f, 62f, "Celulares"),
19          "cel2" to Quadro(255f, 62f, "Celulares"),
20          "nginx" to Quadro(200f, 160f, "Nginx"),
21          "api1" to Quadro(110f, 252f, "API 1"),
22          "mq" to Quadro(200f, 252f, "RabbitMQ"),
23          "api2" to Quadro(290f, 252f, "API 2"),
24          "wk" to Quadro(200f, 338f, "Worker"),
25          "col" to Quadro(46f, 424f, "Coletor"),
```

```
        "db" to Quadro(200f, 424f, "PostgreSQL"),
    )
    private val metade = 34f to 28f
    private val linhas = listOf(
        linha(46f, 90f, 46f, 396f), linha(145f, 90f, 180f, 132f), linha(255f, 90f, 220f, 132f),
        linha(180f, 188f, 125f, 224f), linha(220f, 188f, 275f, 224f), linha(166f, 252f, 144f, 252f), linha(234f, 252f, 256f, 252f),
        linha(200f, 310f, 200f, 280f), linha(200f, 396f, 200f, 366f), linha(110f, 280f, 170f, 396f), linha(290f, 280f, 230f, 396f), linha(80f, 424f, 166f, 424f),
    )
    // Caminho que cada mensagem percorre; celular e API conversam sempre passando pelo Nginx.
    private val rotas: Map<Pair<String, String>, Path> = mapOf(
        ("semil" to "col") to linhas[0], ("col" to "db") to linhas[11], ("db" to "wk") to linhas[8], ("wk" to "mq") to linhas[7],
        ("api1" to "db") to linhas[9], ("api2" to "db") to linhas[10], ("mq" to "api1") to linhas[5], ("mq" to "api2") to linhas[6],
        ("cel1" to "api1") to Path().apply { moveTo(145f, 90f); lineTo(200f, 160f); lineTo(125f, 224f) },
        ("cel2" to "api2") to Path().apply { moveTo(255f, 90f); lineTo(200f, 160f); lineTo(275f, 224f) },
    )
    private fun linha(x1: Float, y1: Float, x2: Float, y2: Float) = Path().apply { moveTo(x1, y1); lineTo(x2, y2) }

    var saude: Map<String, Saude> = emptyMap(); set(valor) { field = valor; invalidate() }
    var voce: String? = null; set(valor) { field = valor; invalidate() }
    var rotulosCelulares: Map<String, String> = emptyMap(); set(valor) { field = valor; invalidate() }
    var rotuloServidor = "VPS · SEDDA.COM.BR"; set(valor) { field = valor; invalidate() }

    private class Pacote(val medida: PathMeasure, val invertido: Boolean, val color: Int)
    private var pacotes: List<Pacote> = emptyList()
    private var inicio = 0L
    private var aoTerminar: (() -> Unit)? = null
    private val duracao = 800L

    private val density = resources.displayMetrics.density
    private val tintaLigacao = Paint(Paint.ANTI_ALIAS_FLAG).apply { style = Paint.Style.STROKE; color = Color.parseColor("#CBD5E1"); strokeCap = Paint.Cap.ROUND }
    private val zona = Paint(Paint.ANTI_ALIAS_FLAG)
    private val bordaZona = Paint(Paint.ANTI_ALIAS_FLAG).apply { style = Paint.Style.STROKE; color = Paleta.linha }
    private val preenchimento = Paint(Paint.ANTI_ALIAS_FLAG)
    private val borda = Paint(Paint.ANTI_ALIAS_FLAG).apply { style = Paint.Style.STROKE }
    private val text = Paint(Paint.ANTI_ALIAS_FLAG).apply { textAlign = Paint.Align.CENTER; typeface = Typeface.create("sans-serif-medium", Typeface.NORMAL) }
    private val legenda = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Paleta.apagado; letterSpacing = 0.08f; typeface = Typeface.create("sans-serif-medium", Typeface.NORMAL) }
    private val bolinha = Paint(Paint.ANTI_ALIAS_FLAG)
    private val verde = Color.parseColor("#16A34A")

    // Anima uma mensagem em cada ligação ao mesmo tempo (ex.: o broker entregando para as duas réplicas).
    fun animar(saltos: List<Pair<String, String>>, color: Int, concluir: () -> Unit) {
        pacotes = saltos.mapNotNull { (from, to) ->
            val ida = rotas[from to to]; val volta = rotas[to to from]
            (ida ?: volta)?.let { Pacote(PathMeasure(it, false), ida == null, color) }
        }
        if (pacotes.isEmpty()) { concluir(); return }
        aoTerminar = concluir; inicio = SystemClock.uptimeMillis(); postInvalidateOnAnimation()
    }
    fun stop() { pacotes = emptyList(); aoTerminar = null; invalidate() }

    override fun onMeasure(widthMeasureSpec: Int, heightMeasureSpec: Int) {
        val w = MeasureSpec.getSize(widthMeasureSpec)
        setMeasuredDimension(w, (w * 474f / 340f).toInt())
    }
    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        val s = width / 340f
        val unidade = density / s
        canvas.save(); canvas.scale(s, s)
        // Duas regiões: a internet (fora do nosso controle) e a VPS (os processos do sistema).
        legenda.textSize = 10f * unidade
        canvas.drawText("INTERNET", 12f, 20f, legenda)
        zona.color = Paleta.fundo; bordaZona.strokeWidth = 1f * unidade
        val areaVps = RectF(4f, 108f, 336f, 466f)
        canvas.drawRoundRect(areaVps, 16f, 16f, zona); canvas.drawRoundRect(areaVps, 16f, 16f, bordaZona)
        Icones.draw(canvas, "servers", 22f, 124f, 13f, Paleta.apagado)
        canvas.drawText(rotuloServidor, 34f, 128f, legenda)

        tintaLigacao.strokeWidth = 1.6f * unidade
        linhas.forEach { canvas.drawPath(it, tintaLigacao) }

        text.textSize = 10.5f * unidade
        for ((key, n) in processos) {
            val estado = saude[key] ?: Saude.DESCONHECIDA
            val meu = voce?.let { key == it || key == chaveCelulares(it) } == true
            val caixa = RectF(n.x - metade.first, n.y - metade.second, n.x + metade.first, n.y + metade.second)
```

```
            preenchimento.color = if (estado == Saude.FORA_DO_AR) Color.parseColor("#FEF2F2") else Color.WHITE
            borda.color = when { estado == Saude.FORA_DO_AR -> Paleta.perigo; meu -> Paleta.destaque; else -> Paleta.linha }
            borda.strokeWidth = (if (meu) 2f else 1f) * unidade
            borda.pathEffect = if (estado == Saude.FORA_DO_AR) DashPathEffect(floatArrayOf(5f, 4f), 0f) else null
            canvas.drawRoundRect(caixa, 12f, 12f, preenchimento); canvas.drawRoundRect(caixa, 12f, 12f, borda)
            val tom = when (estado) { Saude.FORA_DO_AR -> Paleta.perigo; Saude.NO_AR -> Paleta.destaque; Saude.DESCONHECIDA -> Paleta.apagado }
            Icones.draw(canvas, iconeDe(key), n.x, n.y - 7f, 22f, tom)
            text.color = if (estado == Saude.FORA_DO_AR) Paleta.perigo else Paleta.tinta
            canvas.drawText(rotulosCelulares[key] ?: n.rotulo, n.x, n.y + 19f, text)
            // Sinal de vida: verde quando o processo respondeu há pouco, vermelho quando sumiu.
            if (estado != Saude.DESCONHECIDA) {
                bolinha.color = Color.WHITE; canvas.drawCircle(caixa.right - 7f, caixa.top + 7f, 5f, bolinha)
                bolinha.color = if (estado == Saude.NO_AR) verde else Paleta.perigo;
canvas.drawCircle(caixa.right - 7f, caixa.top + 7f, 3.4f, bolinha)
            }
        }

        val k = ((SystemClock.uptimeMillis() - inicio).toFloat() / duracao).coerceIn(0f, 1f)
        val suavizado = k * k * (3 - 2 * k)
        val ponto = FloatArray(2)
        for (p in pacotes) {
            p.medida.getPosTan((if (p.invertido) 1f - suavizado else suavizado) * p.medida.length, ponto, null)
            bolinha.color = Color.WHITE; canvas.drawCircle(ponto[0], ponto[1], 8f, bolinha)
            bolinha.color = p.color; canvas.drawCircle(ponto[0], ponto[1], 6f, bolinha)
        }
        canvas.restore()
        if (pacotes.isNotEmpty()) {
            if (k < 1f) postInvalidateOnAnimation()
            else { pacotes = emptyList(); val concluir = aoTerminar; aoTerminar = null; invalidate(); concluir?.invoke() }
        }
    }
    @SuppressLint("ClickableViewAccessibility")
    override fun onTouchEvent(evento: MotionEvent): Boolean {
        if (evento.action == MotionEvent.ACTION_DOWN) return true
        if (evento.action != MotionEvent.ACTION_UP) return false
        val s = width / 340f; val x = evento.x / s; val y = evento.y / s
        processos.entries.firstOrNull { (_, n) -> kotlin.math.abs(n.x - x) <= metade.first + 6f &&
kotlin.math.abs(n.y - y) <= metade.second + 6f }
            ?.let { (key, _) -> performClick(); aoTocarProcesso(key) }
        return true
    }
    override fun performClick(): Boolean { super.performClick(); return true }

    companion object {
        fun iconeDe(key: String) = when (key) {
            "semi1" -> "globe"; "cel1", "cel2" -> "phone"; "nginx" -> "balancer"; "api1", "api2" -> "server"
            "mq" -> "queue"; "wk" -> "gear"; "col" -> "download"; "db" -> "database"; else -> "bolt"
        }
    }
}
```

android/app/src/main/java/br/com/viaagora/PainelMapa.kt

```
package br.com.viaagora

import android.content.Context
import android.graphics.*
import android.os.Bundle
import android.view.Gravity
import android.widget.FrameLayout
import org.maplibre.android.MapLibre
import org.maplibre.android.camera.CameraUpdateFactory
import org.maplibre.android.geometry.LatLng
import org.maplibre.android.geometry.LatLngBounds
import org.maplibre.android.maps.*
import org.maplibre.android.style.sources.GeoJsonSource
import org.maplibre.android.style.layers.CircleLayer
import org.maplibre.android.style.layers.LineLayer
import org.maplibre.android.style.layers.Property
import org.maplibre.android.style.layers.PropertyFactory.*
import org.maplibre.android.style.expressions.Expression
import org.maplibre.geojson.Feature
import org.maplibre.geojson.FeatureCollection
import org.maplibre.geojson.LineString
import org.maplibre.geojson.Point

// Mapa (MapLibre + OpenStreetMap) com os terminais coloridos pela espera, as rotas das balsas e os relatos.
data class MarcadorMapa(val id: String, val latitude: Double, val longitude: Double, val color: String, val terminal: Boolean)
data class RotaMapa(val pontos: List<Pair<Double, Double>>, val color: String)

```

```
@Suppress("DEPRECATION")
class PainelMapa(context: Context, estadoSalvo: Bundle?, private val aoTocarMarcador: (String) -> Unit,
    private val aoEscolherPonto: (Double, Double) -> Unit, private val aoLocalizar: () -> Unit) :
FrameLayout(context) {
    private val vistaMapa: MapView
    private var mapa: MapLibreMap? = null
    private var itens: List<MarcadorMapa> = emptyList()
    private var rotas: List<RotaMapa> = emptyList()
    private var fonte: GeoJsonSource? = null
    private var fonteRotas: GeoJsonSource? = null
    private val ui = Ui(context)
    private val aviso = ui.text("Carregando mapa…", 13f, Paleta.tinta)
    init {
        setBackgroundColor(Color.parseColor("#E8EEF2"))
        MapLibre.getInstance(context)
        vistaMapa = MapView(context)
        addView(vistaMapa, LayoutParams(LayoutParams.MATCH_PARENT, LayoutParams.MATCH_PARENT))
        vistaMapa.onCreate(estadoSalvo)
        aviso.background = ui.arredondado(Color.WHITE, 12)
        aviso.setPadding(context.dp(12), context.dp(8), context.dp(12), context.dp(8))
        addView(aviso, LayoutParams(LayoutParams.WRAP_CONTENT, LayoutParams.WRAP_CONTENT, Gravity.TOP or
Gravity.START).apply { setMargins(context.dp(10),context.dp(10),context.dp(10),0) })
        val localizar = ui.text("◎", 28f, Paleta.tinta).apply {
            gravity = Gravity.CENTER; background = ui.arredondado(Color.WHITE, 24); elevation =
context.dp(3).toFloat()
            contentDescription = "Minha posição"; isClickable = true; setOnClickListener { aoLocalizar() }
        }
        addView(localizar, LayoutParams(context.dp(48), context.dp(48), Gravity.TOP or Gravity.END).apply {
setMargins(0,context.dp(12),context.dp(12),0) })
        vistaMapa.addOnDidFailLoadingMapListener { aviso.visibility = VISIBLE; aviso.text = "Mapa indisponível
sem internet" }
        vistaMapa.getMapAsync { carregado ->
            mapa = carregado
            carregado.uiSettings.isCompassEnabled = false
            carregado.uiSettings.isLogoEnabled = false
            carregado.uiSettings.isAttributionEnabled = true
            carregado.cameraPosition =
org.maplibre.android.camera.CameraPosition.Builder().target(LatLng(LAT_INICIAL,LNG_INICIAL)).zoom(10.4).build()
            carregado.setStyle(Style.Builder().fromJson(ESTILO)) { style ->
                // Linha de cada travessia (traçado da balsa) por baixo dos terminais.
                fonteRotas = GeoJsonSource("ferry-routes",FeatureCollection.fromFeatures(emptyList()))
                style.addSource(fonteRotas!!)
                style.addLayer(LineLayer("ferry-lines","ferry-routes").withProperties(
                    lineColor(Expression.toColor(Expression.get("color"))), lineCap(Property.LINE_CAP_ROUND),
                    lineWidth(Expression.interpolate(Expression.linear(), Expression.zoom(), Expression.stop(9,
2.5f), Expression.stop(15, 6f)))))
                fonte = GeoJsonSource("ferry-points",FeatureCollection.fromFeatures(emptyList()))
                style.addSource(fonte!!)
                // Bolinhas pequenas na visão geral e maiores de perto, para não se sobreporem fora do lugar.
                val raio = Expression.toNumber(Expression.get("radius"))
                style.addLayer(CircleLayer("ferry-markers","ferry-points").withProperties(
                    circleRadius(Expression.interpolate(Expression.linear(), Expression.zoom(),
                        Expression.stop(9, Expression.product(Expression.literal(0.45f), raio)),
Expression.stop(15, raio))),
                    circleColor(Expression.toColor(Expression.get("color"))),
                    circleStrokeColor(Color.WHITE), circleStrokeWidth(2.5f)))
                aviso.visibility = GONE
                // Enquadra todos os terminais, qualquer que seja o tamanho da tela.
                val limites = LatLngBounds.Builder().includes(catalogoTravessias.flatMap { c -> listOf(c.pontoA,
c.pontoB) }.map { LatLng(it.latitude, it.longitude) }).build()
                post { runCatching { carregado.moveCamera(CameraUpdateFactory.newLatLngBounds(limites,
context.dp(56))) } }
                desenharMarcadores()
            }
            carregado.addOnMapLongClickListener { ponto -> aoEscolherPonto(ponto.latitude,ponto.longitude); true
}
            carregado.addOnMapClickListener { ponto ->
                val tela = carregado.projection.toScreenLocation(ponto)
                val raio = context.dp(24).toFloat()
                val feicoes = carregado.queryRenderedFeatures(RectF(tela.x-raio,tela.y-
raio,tela.x+raio,tela.y+raio),"ferry-markers")
                val id = feicoes.firstOrNull()?.getStringProperty("id")
                if(id!=null){aoTocarMarcador(id);true}else false
            }
        }
    }
    fun atualizar(dados: List<MarcadorMapa>, linhas: List<RotaMapa>) { itens = dados; rotas = linhas;
desenharMarcadores() }
    fun centro(): Pair<Double, Double> {
        val target = mapa?.cameraPosition?.target
        return Pair(target?.latitude ?: LAT_INICIAL, target?.longitude ?: LNG_INICIAL)
    }
    fun focar(latitude: Double, longitude: Double) {
mapa?.animateCamera(CameraUpdateFactory.newLatLngZoom(LatLng(latitude,longitude),15.0)) }
    private fun desenharMarcadores() {
```

```
        fonteRotas?.setGeoJson(FeatureCollection.fromFeatures(rotas.filter { it.pontos.size >= 2 }.map { rota ->
            Feature.fromGeometry(LineString.fromLngLats(rota.pontos.map { Point.fromLngLat(it.second, it.first)
})).apply { addStringProperty("color", rota.color) }
        }))
        fonte?.setGeoJson(FeatureCollection.fromFeatures(itens.map { item ->
            Feature.fromGeometry(Point.fromLngLat(item.longitude,item.latitude)).apply {
                addStringProperty("id",item.id)
                addStringProperty("color",item.color)
                addNumberProperty("radius",if(item.terminal)14f else 9f)
            }
        }))
    }
    fun start() = vistaMapa.onStart()
    fun resume() = vistaMapa.onResume()
    fun pause() = vistaMapa.onPause()
    fun stop() = vistaMapa.onStop()
    fun destroy() = vistaMapa.onDestroy()
    fun lowMemory() = vistaMapa.onLowMemory()
    fun save(bundle: Bundle) = vistaMapa.onSaveInstanceState(bundle)
    companion object {
        private const val LAT_INICIAL = -23.925
        private const val LNG_INICIAL = -46.215
        private const val ESTILO =
"""{"version":8,"name":"ViaAgora","sources":{"osm":{"type":"raster","tiles":["https://tile.openstreetmap.org/{z}/{x}/
{y}.png"],"tileSize":256,"attribution":"© OpenStreetMap
contributors"}},"layers":[{"id":"background","type":"background","paint":{"background-
color":"#E8EEF2"}},{"id":"streets","type":"raster","source":"osm"}]}"""
    }
}
```

android/app/src/main/java/br/com/viaagora/GraficoHistorico.kt

```
package br.com.viaagora

import android.content.Context
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.view.View

// Barras com a espera média por hora do dia, uma cor para cada lado. A melhor hora (entre 6h e 22h) de cada lado
fica em verde.
class GraficoHistorico(context: Context, private val historico: Historico) : View(context) {
    private val barraA = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Paleta.marinho }
    private val barraB = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Paleta.destaque }
    private val melhor = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Color.parseColor(NivelEspera.BAIXA.color) }
    private val rotulo = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Paleta.apagado; textSize =
context.dp(10).toFloat(); textAlign = Paint.Align.CENTER }
    private val grade = Paint().apply { color = Paleta.linha; strokeWidth = context.dp(1).toFloat() }
    init {
        contentDescription = "Espera média por hora. " + historico.porHora.joinToString("; ") { "${it.hora}h:
${it.esperaA} e ${it.esperaB} minutos" }
    }
    override fun onMeasure(widthMeasureSpec: Int, heightMeasureSpec: Int) {
        setMeasuredDimension(MeasureSpec.getSize(widthMeasureSpec), context.dp(170))
    }
    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        val bottom = height - context.dp(18f)
        val top = context.dp(8f)
        val max = (historico.porHora.maxOfOrNull { maxOf(it.esperaA, it.esperaB) } ?: 0).coerceAtLeast(10)
        val faixa = width / 24f
        val barra = faixa * 0.36f
        canvas.drawLine(0f, bottom, width.toFloat(), bottom, grade)
        for (h in historico.porHora) {
            val x = h.hora * faixa + faixa * 0.12f
            val alturaA = (bottom - top) * h.esperaA / max
            val alturaB = (bottom - top) * h.esperaB / max
            canvas.drawRect(x, bottom - alturaA, x + barra, bottom, if (h.hora == historico.melhorA) melhor else
barraA)
            canvas.drawRect(x + barra, bottom - alturaB, x + barra * 2, bottom, if (h.hora == historico.melhorB)
melhor else barraB)
        }
        for (hora in 0..23 step 3) canvas.drawText("${hora}h", hora * faixa + faixa / 2, height - context.dp(4f),
rotulo)
    }
    private fun Context.dp(valor: Float) = valor * resources.displayMetrics.density
}
```

android/app/src/main/java/br/com/viaagora/ComponentesUi.kt

```
package br.com.viaagora

import android.app.Dialog
```

```
import android.content.Context
import android.graphics.Color
import android.graphics.Typeface
import android.graphics.drawable.ColorDrawable
import android.graphics.drawable.GradientDrawable
import android.text.SpannableStringBuilder
import android.text.Spanned
import android.text.style.AbsoluteSizeSpan
import android.text.style.ForegroundColorSpan
import android.view.Gravity
import android.view.View
import android.view.ViewGroup
import android.view.WindowManager
import android.widget.*

// Componentes visuais usados em todas as telas: cores, textos, botões e o painel que sobe de baixo.
// Converte dp (unidade que não depende da densidade da tela) em pixels.
fun Context.dp(valor: Int) = (valor * resources.displayMetrics.density).toInt()
// Cores do aplicativo.
object Paleta {
    val marinho = Color.parseColor("#0B2E4F")
    val tinta = Color.parseColor("#0F172A")
    val destaque = Color.parseColor("#0B7A8A")
    val apagado = Color.parseColor("#64748B")
    val fundo = Color.parseColor("#F5F7FA")
    val suave = Color.parseColor("#EEF2F6")
    val linha = Color.parseColor("#E2E8F0")
    val alerta = Color.parseColor("#B45309")
    val perigo = Color.parseColor("#DC2626")
}
// Fábrica de componentes de tela com o estilo do app.
class Ui(val context: Context) {
    fun arredondado(color: Int, raio: Int = 16, borda: Int? = null) = GradientDrawable().apply {
        setColor(color); cornerRadius = context.dp(raio).toFloat()
        if (borda != null) setStroke(context.dp(1), borda)
    }
    fun text(valor: CharSequence, size: Float = 15f, color: Int = Paleta.tinta, negrito: Boolean = false) =
TextView(context).apply {
        text = valor; textSize = size; setTextColor(color)
        if (negrito) typeface = Typeface.create("sans-serif-medium", Typeface.NORMAL)
        includeFontPadding = false
    }
    fun coluna() = LinearLayout(context).apply { orientation = LinearLayout.VERTICAL }
    fun linha() = LinearLayout(context).apply { orientation = LinearLayout.HORIZONTAL; gravity =
Gravity.CENTER_VERTICAL }
    fun botao(rotulo: String, principal: Boolean = false, aoTocar: () -> Unit) = TextView(context).apply {
        text = rotulo; textSize = 15f; gravity = Gravity.CENTER
        typeface = Typeface.create("sans-serif-medium", Typeface.NORMAL)
        setTextColor(if (principal) Color.WHITE else Paleta.tinta)
        background = arredondado(if (principal) Paleta.destaque else Paleta.suave, 14)
        minHeight = context.dp(48); setPadding(context.dp(16), 0, context.dp(16), 0)
        isClickable = true; isFocusable = true; contentDescription = rotulo
        setOnClickListener { aoTocar() }
    }
    fun etiqueta(rotulo: String, selecionada: Boolean, aoTocar: () -> Unit) = text(rotulo, 14f, if (selecionada)
Color.WHITE else Paleta.tinta, selecionada).apply {
        gravity = Gravity.CENTER; minHeight = context.dp(40)
        setPadding(context.dp(16), 0, context.dp(16), 0)
        background = arredondado(if (selecionada) Paleta.tinta else Paleta.suave, 20)
        isClickable = true; isFocusable = true; contentDescription = rotulo
        setOnClickListener { aoTocar() }
    }
    fun espaco(height: Int) = View(context).apply { layoutParams = LinearLayout.LayoutParams(1,
context.dp(height)) }
    fun entrada(dica: String, valor: String = "") = EditText(context).apply {
        hint = dica; setText(valor); textSize = 15f; setTextColor(Paleta.tinta)
        setHintTextColor(Paleta.apagado); background = arredondado(Paleta.suave, 12)
        setPadding(context.dp(14), context.dp(12), context.dp(14), context.dp(12))
    }
    fun rotulo(valor: String) = text(valor, 13f, Paleta.apagado, true).apply { setPadding(0, context.dp(16), 0,
context.dp(8)) }
    // "15 min" com o número grande e a unidade pequena.
    fun minutos(valor: Int?, color: Int, size: Int = 32): CharSequence {
        if (valor == null) return "—"
        val numero = valor.toString()
        return SpannableStringBuilder("$numero min").apply {
            setSpan(AbsoluteSizeSpan(size, true), 0, numero.length, Spanned.SPAN_EXCLUSIVE_EXCLUSIVE)
            setSpan(AbsoluteSizeSpan(size / 2, true), numero.length, length, Spanned.SPAN_EXCLUSIVE_EXCLUSIVE)
            setSpan(ForegroundColorSpan(color), 0, length, Spanned.SPAN_EXCLUSIVE_EXCLUSIVE)
        }
    }
    fun grande(valor: String, color: Int, size: Int): CharSequence = SpannableStringBuilder(valor).apply {
        setSpan(AbsoluteSizeSpan(size, true), 0, length, Spanned.SPAN_EXCLUSIVE_EXCLUSIVE)
        setSpan(ForegroundColorSpan(color), 0, length, Spanned.SPAN_EXCLUSIVE_EXCLUSIVE)
    }
```

```
    // Painel que sobe da parte de baixo da tela, no lugar das caixas de diálogo.
    fun painel(titulo: String, conteudo: View): Dialog {
        val dialog = Dialog(context)
        val caixa = coluna().apply {
            setPadding(context.dp(24), context.dp(10), context.dp(24), context.dp(24))
            background = GradientDrawable().apply {
                setColor(Color.WHITE)
                val r = context.dp(24).toFloat(); cornerRadii = floatArrayOf(r, r, r, r, 0f, 0f, 0f, 0f)
            }
        }
        caixa.addView(View(context).apply { background = arredondado(Paleta.linha, 2) },
            LinearLayout.LayoutParams(context.dp(40), context.dp(4)).apply { gravity =
Gravity.CENTER_HORIZONTAL; bottomMargin = context.dp(18) })
        caixa.addView(text(titulo, 20f, Paleta.tinta, true))
        caixa.addView(espaco(4))
        caixa.addView(conteudo)
        dialog.setContentView(ScrollView(context).apply { addView(caixa) })
        dialog.window?.apply {
            setLayout(ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.WRAP_CONTENT)
            setGravity(Gravity.BOTTOM)
            setBackgroundDrawable(ColorDrawable(Color.TRANSPARENT))
            setWindowAnimations(android.R.style.Animation_InputMethod)
            setSoftInputMode(WindowManager.LayoutParams.SOFT_INPUT_ADJUST_RESIZE)
        }
        return dialog
    }
}

// Linha de opções em que só uma fica selecionada.
class Escolha(private val ui: Ui, rotulos: List<String>, inicial: Int = 0, private val aoMudar: (Int) -> Unit =
{}) {
    private val linha = ui.linha()
    val view = HorizontalScrollView(ui.context).apply { isHorizontalScrollBarEnabled = false; addView(linha) }
    private var rotulos = rotulos
    var selecionada = inicial
        private set
    init { atualizarTela() }
    fun trocarRotulos(values: List<String>) { rotulos = values; atualizarTela() }
    private fun atualizarTela() {
        linha.removeAllViews()
        rotulos.forEachIndexed { indice, rotulo ->
            linha.addView(ui.etiqueta(rotulo, indice == selecionada) { selecionada = indice; atualizarTela();
aoMudar(indice) },
                LinearLayout.LayoutParams(LinearLayout.LayoutParams.WRAP_CONTENT, ui.context.dp(40)).apply {
marginEnd = ui.context.dp(8) })
        }
    }
}
```

android/app/src/main/java/br/com/viaagora/Icones.kt

```
package br.com.viaagora

import android.graphics.*
import android.graphics.drawable.Drawable
import androidx.core.graphics.PathParser

// Ícones de traço (grade 24 x 24), os mesmos desenhos do site.
object Icones {
    private val dados = mapOf(
        "phone" to "M8.5,2h7a2.5,2.5 0,0 1,2.5,2.5v15a2.5,2.5 0,0 1,-2.5,2.5h-7a2.5,2.5 0,0 1,-2.5,-2.5v-
15a2.5,2.5 0,0 1,2.5,-2.5z M10.5,5h3 M11,18.5h2",
        "server" to "M4.5,4h15a1.5,1.5 0,0 1,1.5,1.5v4a1.5,1.5 0,0 1,-1.5,1.5h-15a1.5,1.5 0,0 1,-1.5,-1.5v-
4a1.5,1.5 0,0 1,1.5,-1.5z " +
            "M4.5,13h15a1.5,1.5 0,0 1,1.5,1.5v4a1.5,1.5 0,0 1,-1.5,1.5h-15a1.5,1.5 0,0 1,-1.5,-1.5v-4a1.5,1.5 0,0
1,1.5,-1.5z M7,7.5h0.01 M7,16.5h0.01 M11,7.5h6 M11,16.5h6",
        "servers" to "M4.5,3h15a1.5,1.5 0,0 1,1.5,1.5v2.5a1.5,1.5 0,0 1,-1.5,1.5h-15a1.5,1.5 0,0 1,-1.5,-1.5v-
2.5a1.5,1.5 0,0 1,1.5,-1.5z " +
            "M4.5,9.25h15a1.5,1.5 0,0 1,1.5,1.5v2.5a1.5,1.5 0,0 1,-1.5,1.5h-15a1.5,1.5 0,0 1,-1.5,-1.5v-
2.5a1.5,1.5 0,0 1,1.5,-1.5z " +
            "M4.5,15.5h15a1.5,1.5 0,0 1,1.5,1.5v2.5a1.5,1.5 0,0 1,-1.5,1.5h-15a1.5,1.5 0,0 1,-1.5,-1.5v-
2.5a1.5,1.5 0,0 1,1.5,-1.5z " +
            "M6.5,5.75h0.01 M6.5,12h0.01 M6.5,18.25h0.01 M11,5.75h6 M11,12h6 M11,18.25h6",
        "balancer" to "M7,12a2,2 0 1,1 -4,0a2,2 0 1,1 4,0z M7,12h3l4,-5.5h4 M10,12l4,5.5h4 M16.5,4.5l2,2 -2,2
M16.5,15.5l2,2 -2,2",
        "database" to "M4.5,5.5a7.5,2.8 0 1,0 15,0a7.5,2.8 0 1,0 -15,0z M4.5,5.5v13c0,1.55 3.36,2.8 7.5,2.8s7.5,-
1.25 7.5,-2.8v-13 M4.5,12c0,1.55 3.36,2.8 7.5,2.8s7.5,-1.25 7.5,-2.8",
        "gear" to
"M19.11,9.32L21.8,10.01L21.8,13.99L19.11,14.68L18.92,15.14L20.33,17.53L17.53,20.33L15.14,18.92L14.68,19.11L13.99,21.8
L10.01,21.8L9.32,19.11" +
"L8.86,18.92L6.47,20.33L3.67,17.53L5.08,15.14L4.89,14.68L2.2,13.99L2.2,10.01L4.89,9.32L5.08,8.86L3.67,6.47L6.47,3.67L
8.86,5.08L9.32,4.89L10.01,2.2" +
            "L13.99,2.2L14.68,4.89L15.14,5.08L17.53,3.67L20.33,6.47L18.92,8.86z M15,12a3,3 0 1,1 -6,0a3,3 0 1,1
6,0z",
```

```
        "sliders" to "M4,7h10 M18,7h2 M18,7a2,2 0 1,1 -4,0a2,2 0 1,1 4,0z M4,17h2 M10,17h10 M10,17a2,2 0 1,1 -4,0a2,2 0 1,1 4,0z",
        "queue" to "M5,6h14a2,2 0 0,1 2,2v8a2,2 0 0,1 -2,2h-14a2,2 0 0,1 -2,-2v-8a2,2 0 0,1 2,-2z M3.5,7.5l8.5,6 8.5,-6 M6,3h12",
        "download" to "M12,3v11.5 M7.5,10l4.5,4.5 4.5,-4.5 M4,19.5h16",
        "globe" to "M21,12a9,9 0 1,1 -18,0a9,9 0 1,1 18,0z M16,12a4,9 0 1,1 -8,0a4,9 0 1,1 8,0z M3,12h18",
        "bolt" to "M13,2.5L4.5,13.5H11l-1,8 8.5,-11H12z",
        "clock" to "M21,12a9,9 0 1,1 -18,0a9,9 0 1,1 18,0z M12,7v5l3,2",
        "repeat" to "M17,2.5l3,3 -3,3 M4,11.5v-1a5,5 0 0,1 5,-5h11 M7,21.5l-3,-3 3,-3 M20,12.5v1a5,5 0 0,1 -5,5H4",
    )
    private val cache = mutableMapOf<String, Path>()
    fun caminho(nome: String): Path = cache.getOrPut(nome) { PathParser.createPathFromPathData(dados.getValue(nome)) }
    private val pincel = Paint(Paint.ANTI_ALIAS_FLAG).apply { style = Paint.Style.STROKE; strokeCap = Paint.Cap.ROUND; strokeJoin = Paint.Join.ROUND }

    // Desenha o ícone com o centro em (cx, cy) e o lado "size", nas unidades atuais do canvas.
    fun draw(canvas: Canvas, nome: String, cx: Float, cy: Float, size: Float, color: Int) {
        val k = size / 24f
        pincel.color = color; pincel.strokeWidth = 1.7f
        canvas.save(); canvas.translate(cx - size / 2, cy - size / 2); canvas.scale(k, k)
        canvas.drawPath(caminho(nome), pincel)
        canvas.restore()
    }
}

// O mesmo ícone como Drawable, para usar ao lado de textos.
class DesenhoIcone(private val nome: String, private val color: Int) : Drawable() {
    override fun draw(canvas: Canvas) {
        val b = bounds
        Icones.draw(canvas, nome, b.exactCenterX(), b.exactCenterY(), minOf(b.width(), b.height()).toFloat(), color)
    }
    override fun setAlpha(alpha: Int) {}
    override fun setColorFilter(colorFilter: ColorFilter?) {}
    @Deprecated("Deprecated in Java") override fun getOpacity() = PixelFormat.TRANSLUCENT
}
```